# 星绽操作系统 dm-crypt/dm-verity 设计与实现技术文档

全国操作系统设计赛——功能挑战赛道

参赛队伍：Go_Go_Go
高校：华南师范大学
成员：张乐桁、陈韵、潘利源

2026 年 8 月 11 日

# 目录


**1 项目概述与背景** **6**

1.1 项目背景 . . . . . 6
1.2 项目目标 . . . . . 6
1.3 双目标架构创新 . . . . . 7
1.4 系统架构概述 . . . . . 7
1.5 文档结构 . . . . . 8
1.6 小结 . . . . . 8

**2 系统架构设计** **9**

2.1 设计原则 . . . . . 9
2.1.1 核心抽象：BlockDevice Trait . . . . . 9
2.1.2 可组合的映射器链模式 . . . . . 10
2.2 核心模块分解 . . . . . 10
2.3 跨环境可移植性 . . . . . 11
2.4 数据流架构 . . . . . 12
2.5 线程安全保证 . . . . . 13
2.6 性能可观测性设计 . . . . . 13
2.7 运行时控制面（Linux 兼容） . . . . . 14
2.8 小结 . . . . . 14

**3 dm-verity 完整性校验详细设计** **15**

3.1 概述 . . . . . 15
3.2 架构组成 . . . . . 15
3.3 默克尔树构建机制 . . . . . 16
3.3.1 构建流程 . . . . . 16
3.3.2 内存与磁盘布局 . . . . . 16
3.4 单块校验路径 . . . . . 17
3.5 只读强制策略 . . . . . 18
3.6 启动参数解析 . . . . . 18
3.6.1 支持的参数 . . . . . 18

3.6.2 解析流程 . . . 18
3.6.3 内核自检机制 . . . 19
3.7 认证后哈希缓存与优化 . . . 19
3.8 校验失败处理 . . . 20
3.9 小结 . . . 20

**4 dm-crypt 透明加密详细设计 21**
4.1 概述 . . . 21
4.2 系统架构与数据流 . . . 21
4.3 核心组件 . . . 22
4.3.1 DmCryptTable——加密目标配置表 . . . 22
4.3.2 Aes256XtsCipher——核心密码引擎 . . . 22
4.3.3 DmCryptDevice——主设备映射目标 . . . 23
4.4 实现细节：扇区转换与 I/O 路径 . . . 23
4.5 Linux 兼容性验证 . . . 24
4.5.1 兼容性对比 . . . 24
4.5.2 交叉验证方法 . . . 24
4.6 根文件系统加密方案 . . . 24
4.7 小结 . . . 25

**5 性能优化策略 26**
5.1 概述 . . . 26
5.2 性能评估框架 . . . 26
5.2.1 关键性能指标 . . . 26
5.2.2 正式测试合同 . . . 26
5.2.3 观察者开销校准 . . . 27
5.3 哈希树优化（dm-verity） . . . 27
5.3.1 认证后哈希块缓存 . . . 27
5.3.2 已认证路径捷径 . . . 28
5.3.3 完成 BIO 数据复用 . . . 28
5.4 I/O 吞吐优化（dm-crypt） . . . 28
5.4.1 有界临时缓冲池（Scratch Pool） . . . 28
5.4.2 AES-NI 硬件加速 . . . 29
5.4.3 完成队列与并发优化（实验性） . . . 29
5.5 启动时间控制 . . . 29
5.5.1 延迟校验策略 . . . 29
5.5.2 校验路径优化 . . . 29
5.6 性能归因与阶段统计 . . . 30

5.7 与 Linux guest 的同条件对照 . . . 31
5.8 小结 . . . 31

**6 测试与性能分析报告 32**
6.1 概述 . . . 32
6.2 测试架构概述 . . . 32
6.3 功能正确性测试 . . . 33
6.3.1 dm-verity 完整性校验测试 . . . 33
6.3.2 dm-crypt 透明加密测试 . . . 34
6.3.3 Linux 兼容性验证 . . . 35
6.3.4 内核端到端功能验证 . . . 35
6.4 性能基准测试 . . . 35
6.4.1 正式测试合同 . . . 35
6.4.2 I/O 性能测试结果 . . . 36
6.4.3 启动开销分析 . . . 37
6.4.4 Linux guest 与 Asterinas 同条件对照 . . . 37
6.5 创新技术验证 . . . 37
6.6 证据可复核性 . . . 38
6.7 小结 . . . 38

**7 合规、来源与 AI 声明 39**
7.1 概述 . . . 39
7.2 代码与文档来源及开源协议说明 . . . 39
7.2.1 引用的开源代码 . . . 39
7.2.2 开源协议合规性声明 . . . 40
7.3 借鉴基础版本与增量贡献说明 . . . 40
7.3.1 借鉴基础 . . . 40
7.3.2 本队增量贡献 . . . 40
7.4 AI 工具使用声明 . . . 41
7.4.1 使用的 AI 工具及大模型 . . . 41
7.4.2 AI 交互记录 . . . 41
7.4.3 验证机制与责任声明 . . . 44
7.5 Git 提交与开发迭代记录 . . . 44
7.5.1 提交概览 . . . 44
7.5.2 关键迭代节点说明 . . . 46
7.6 小结 . . . 46

**8 用户手册与部署指南 47**

8.1 概述 . . . 47
8.2 运行环境准备 . . . 47
8.2.1 宿主机要求 . . . 47
8.2.2 依赖工具安装 . . . 48
8.3 快速开始：用户态演示 . . . 48
8.3.1 克隆仓库与运行测试套件 . . . 48
8.3.2 演示透明加密（dm-crypt） . . . 48
8.3.3 演示完整性校验（dm-verity） . . . 49
8.3.4 演示组合栈（dm-crypt + dm-verity） . . . 49
8.3.5 手动命令示例 . . . 49
8.4 星绽内核集成 . . . 50
8.4.1 准备 Asterinas 源码 . . . 50
8.4.2 应用 Device Mapper 补丁 . . . 50
8.4.3 构建内核 . . . 51
8.4.4 构建受保护的根文件系统镜像 . . . 51
8.5 系统运行与参数配置 . . . 52
8.5.1 QEMU 启动命令 . . . 52
8.5.2 内核启动参数配置 . . . 52
8.6 基准测试与性能验证 . . . 53
8.6.1 正式性能测试（256 MiB 合同） . . . 53
8.6.2 启动时间测量 . . . 54
8.6.3 用户态微基准测试 . . . 54
8.7 高级配置与调优 . . . 54
8.7.1 dm-crypt 临时缓冲池调优 . . . 54
8.7.2 dm-verity 哈希缓存调优 . . . 54
8.7.3 内核阶段统计 . . . 54
8.8 故障排查 . . . 55
8.8.1 常见问题 . . . 55
8.8.2 调试技巧 . . . 55
8.9 小结 . . . 55

**9 研发过程与问题解决记录 57**

9.1 概述 . . . 57
9.2 核心技术难题及解决方法 . . . 57
9.2.1 Rust 所有权与 I/O 队列管理 . . . 57
9.2.2 星绽 OSTD 框架 API 适配 . . . 58
9.2.3 DMA 切片与 BIO 路径验证 . . . 58
9.2.4 根文件系统切换（pivot_root）与映射器生命周期 . . . 58

9.3 内核集成补丁说明 . . . . . 59
9.3.1 补丁 0001：组件集成与目标定义 . . . . . 59
9.3.2 补丁 0002：BIO 路径验证与自检 . . . . . 60
9.3.3 补丁 0003：根文件系统 Pivot_root 切换 . . . . . 60
9.3.4 补丁 0004：Runtime 控制面与 cryptsetup 兼容 . . . . . 61
9.3.5 补丁 0005：性能优化与可配置基准 . . . . . 61
9.3.6 补丁 0006：性能阶段归因与正式基准合同 . . . . . 62
9.4 五阶段渐进式保护路径 . . . . . 63
9.5 实验性优化探索 . . . . . 63
9.5.1 异步 BIO 完成路径 . . . . . 63
9.5.2 完成队列与专用 worker . . . . . 63
9.6 小结 . . . . . 64

**A 附录A：术语表 65**

**B 附录B：dmctl 命令参考 67**

# Chapter 1

# 项目概述与背景

## 1.1 项目背景

星绽操作系统（Asterinas）是一个采用创新性 Framekernel 架构的 Rust 操作系统，通过 OSTD 框架将内核核心组件运行于非特权级，实现了可信计算基（TCB）的最小化。依托 Rust 的所有权与生命周期机制，星绽从根源上规避了内存安全风险，同时在性能关键路径上保持高效执行。

在机密计算（Confidential Computing, CoCo）场景下，星绽计划作为 Guest Kernel 运行于可信虚拟机中。尽管内存数据受 TEE 保护，但持久化存储中的镜像（如 rootfs）仍存储在宿主机的不可信介质上，面临离线篡改、镜像替换、数据窃取等安全威胁。攻击者可借此植入后门或破坏系统完整性，危及整个信任链。

Linux 通过 Device Mapper 提供了 `dm-crypt`（透明数据加密）和 `dm-verity`（只读数据完整性校验）机制[1, 2]，广泛用于全盘加密与安全启动。本赛题要求在星绽 OS 中，利用 Rust 的内存安全优势，自主实现类 `dm-crypt` 与 `dm-verity` 模块，填补星绽生态在存储安全方面的空白[6]。

## 1.2 项目目标

本项目在星绽操作系统上使用 Rust 编程语言实现 `dm-crypt` 与 `dm-verity` 功能，具体技术指标如下：

1. `dm-verity` **基础功能**：构建 SHA-256 哈希树[5]，支持内核启动参数配置（如 `rootfs_verity.scheme=dm-verity rootfs_verity.hash=<hash>`），在系统启动时对根文件系统进行完整性验证。
2. `dm-crypt` **基础功能**：支持透明加解密 I/O 路径，兼容 Linux `aes-xts-plain64` 密码格式[4]，作为后端支撑 `cryptsetup` 对加密卷的创建和管理[3]。
3. **根文件系统加密**：使用 `dm-crypt` 加解密根文件系统，确保持久化数据的机密性。
4. **性能与启动要求**：I/O 吞吐及延迟接近 Linux，`dm-verity` 不显著增加内核启动时

间。

5. **技术探索**：探索基于 Rust OS 的可信存储性能优化技术，如认证后哈希缓存、有界临时缓冲池和硬件加速（AES-NI）。

当前版本已完成上述全部指标的用户态验证和星绽内核集成，并包含可审计的补丁序列和 KVM 五轮基准测试证据。

## 1.3 双目标架构创新

本项目的一个关键创新在于其**双目标架构**：核心逻辑被实现为一个与平台无关的 Rust 库（`dm` crate），该库同时服务于两个目标环境：

- **用户态命令行工具（`dmctl`）**：用于演示、测试和基准测试，能够在宿主机上快速验证加密和完整性机制的正确性。
- **星绽内核集成（`aster-dm`）**：将相同的核心逻辑移植到内核空间，使星绽能够利用 `virtio-blk` 设备作为加密或完整性保护的卷，直接挂载受保护的文件系统（如 ext2）。

这种设计确保了用户态与内核态的逻辑一致性，使得测试和调试更加高效，同时也为后续的性能优化和功能扩展提供了统一的抽象基础。

## 1.4 系统架构概述

系统的核心抽象是 `BlockDevice` trait，它定义了统一的块级 I/O 接口（`read_block` 和 `write_block`）。不同的存储后端（如内存、文件、物理磁盘）均实现该 trait，而加密和完整性校验模块（`DmCryptDevice`、`DmVerityDevice`）则以“装饰器”模式包装下层设备，形成可堆叠的映射器链。

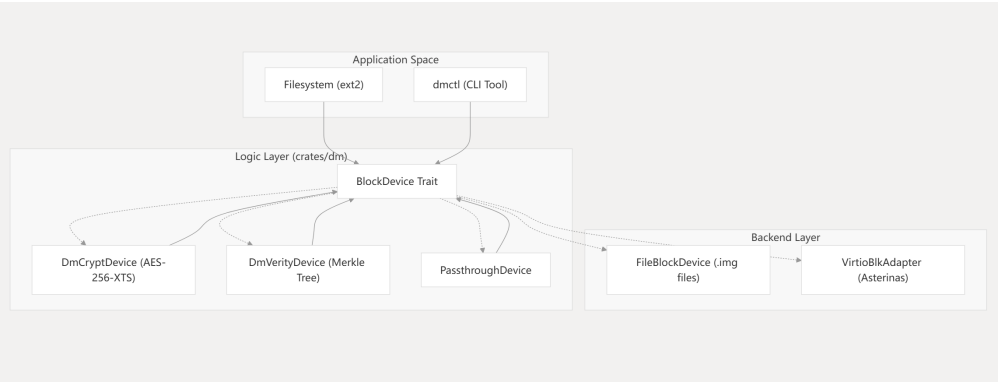



图 1.1: 系统组件关系图

如图所示，文件系统或应用程序与顶层的 `BlockDevice`（例如 `DmCryptDevice`）交互，该设备在执行加密转换后，将 I/O 委托给下一层设备（如 `VirtioBlkAdapter` 或 `FileBlockDevice`），最终到达物理存储介质。

## 1.5 文档结构

本技术文档共分为九章和两个附录，结构如下：

- **第2章系统架构设计**：详细阐述 `BlockDevice` trait、可组合映射器链、模块分解及跨环境可移植性。
- **第3章 `dm-verity` 详细设计**：介绍默克尔树构建、单块校验路径、启动参数解析及只读强制策略。
- **第4章 `dm-crypt` 详细设计**：说明 `DmCryptTable`、AES-256-XTS 密码引擎、扇区算术及 Linux 兼容性验证。
- **第5章性能优化策略**：涵盖认证后哈希缓存、完成 BIO 数据复用、有界临时缓冲池及 AES-NI 硬件加速等优化手段。
- **第6章测试与性能分析报告**：包括功能正确性测试（篡改检测、明文/密文验证）、FIO 基准测试、启动开销分析及与 Linux 的交叉验证。
- **第7章合规、来源与 AI 声明**：独立章节，严格遵循比赛要求，披露代码来源、开源协议及 AI 工具使用情况。
- **第8章用户手册与部署指南**：提供从环境准备到编译、运行、配置的完整操作步骤。
- **第9章研发过程与问题解决记录**：记录核心难题（如 Rust 所有权冲突、DMA 路径验证）及解决思路，并展示 Git 迭代历史。
- **附录 A 术语表**：定义代码库中的关键术语。
- **附录 B `dmctl` 命令参考**：列出所有子命令的签名、示例和实现说明。

## 1.6 小结

本章介绍了项目的背景、目标、双目标架构创新及整体文档结构。后续章节将深入阐述每个模块的设计与实现细节，并通过测试数据验证系统的功能正确性与性能表现。

# Chapter 2

# 系统架构设计

## 2.1 设计原则

本项目的基本设计原则是使用统一的抽象来表示任何能够进行块级 I/O 的实体。这种抽象允许采用“分层”或“可组合”的架构，使得加密和校验逻辑与底层存储介质完全解耦，从而在用户态命令行工具和星绽内核中实现相同的核心逻辑。

### 2.1.1 核心抽象：BlockDevice Trait

`BlockDevice` trait 是库中所有存储操作的基础接口。它定义了用于读写固定大小数据块的同步接口。为了支持跨线程的并发访问，并安全地集成到星绽内核或用户态工具中，该 trait 要求实现 `Send` 和 `Sync` 标记。

该 trait 定义如下：

```
pub trait BlockDevice: Send + Sync {
    fn block_size(&self) -> usize;
    fn num_blocks(&self) -> u64;
    fn read_block(&self, block_id: u64, buf: &mut [u8]) ->
       Result<()>;
    fn write_block(&self, block_id: u64, buf: &[u8]) -> Result
       <()>;
}
```

所有 I/O 方法均返回 `Result<()>`，其中错误被封装在 `DmError` 枚举中。

在 Asterinas 内核中，该 trait 进一步映射到 `aster-block` 组件的 BIO 接口，从而支持真实的 VirtIO 块设备。

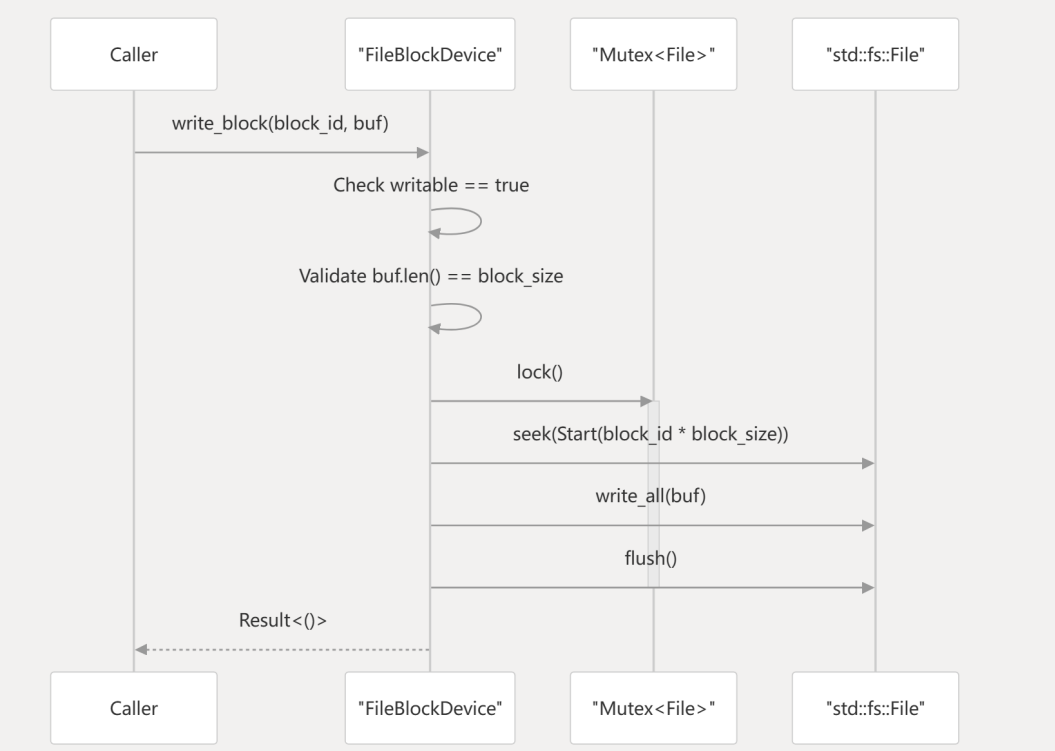



图 2.1: BlockDevice Trait 与后端实现关系图

### 2.1.2 可组合的映射器链模式

该架构采用类似装饰器（Decorator）的设计模式。其中“映射器”（如 `DmCryptDevice` 和 `DmVerityDevice`）既实现了 `BlockDevice` trait，又在其内部消费（包装）另一个实现了 `BlockDevice` 的对象。这允许功能的任意堆叠与组合。

例如，一个典型的栈可能是：

```
let file = FileBlockDevice::open("disk.img")?;
let crypt = DmCryptDevice::new(file, key)?;
let verity = DmVerityDevice::new(crypt, root_hash)?;
```

在星绽内核中，这种组合通过 `DmDevice` 结构体实现，它根据 `DmTarget` 枚举（Passthrough、Crypt、Verity）分发 BIO 请求。

## 2.2 核心模块分解

项目被结构化为离散的模块，以清晰地分离密码学原语、数据结构和设备逻辑。

表 2.1: 核心模块职责划分

| 模块 | 主要职责 | 关键实体 |
|---|---|---|
| block_device | 基础 I/O 抽象与物理/内存后端 | BlockDevice, FileBlockDevice, MemoryBlockDevice |
| dm_crypt | 使用 AES-XTS 的透明加密逻辑 | DmCryptDevice, DmCryptTable |
| dm_verity | 只读完整性校验逻辑 | DmVerityDevice |
| hash_tree | 默克尔树的构建与管理 | HashTree |
| sha256 | 纯 Rust 实现的 SHA-256（无外部依赖） | sha256::sha256 |
| device_mapper | 设备链接（chaining）的基础设施 | PassthroughDevice, DmStackBuilder |
| statistics | I/O 性能计数器与阶段归因 | InstrumentedDevice, IoStatsSnapshot |
| runtime | 运行时 mapper 管理（Linux dm_ioctl 兼容） | RuntimeMapper, DmControlDevice |
| error | 统一错误处理 | DmError, Result |

BlockDevice 实现涵盖了多种后端场景：

- MemoryBlockDevice：基于内存向量的设备，主要用于单元测试和易失性存储。
- FileBlockDevice：由宿主机文件系统上的文件支持的持久化设备，由 dmctl 工具使用。
- PassthroughDevice：将调用直接转发给内部设备的包装器，用于验证映射器链。
- VirtioBlkAdapter：（星绽移植版）将内核特定的块设备句柄适配到 BlockDevice 接口，位于 aster-dm 组件中。

## 2.3 跨环境可移植性

一个核心设计目标是：相同的逻辑同时服务于用户态演示和星绽内核。这是通过严格隔离环境特定代码来实现的。

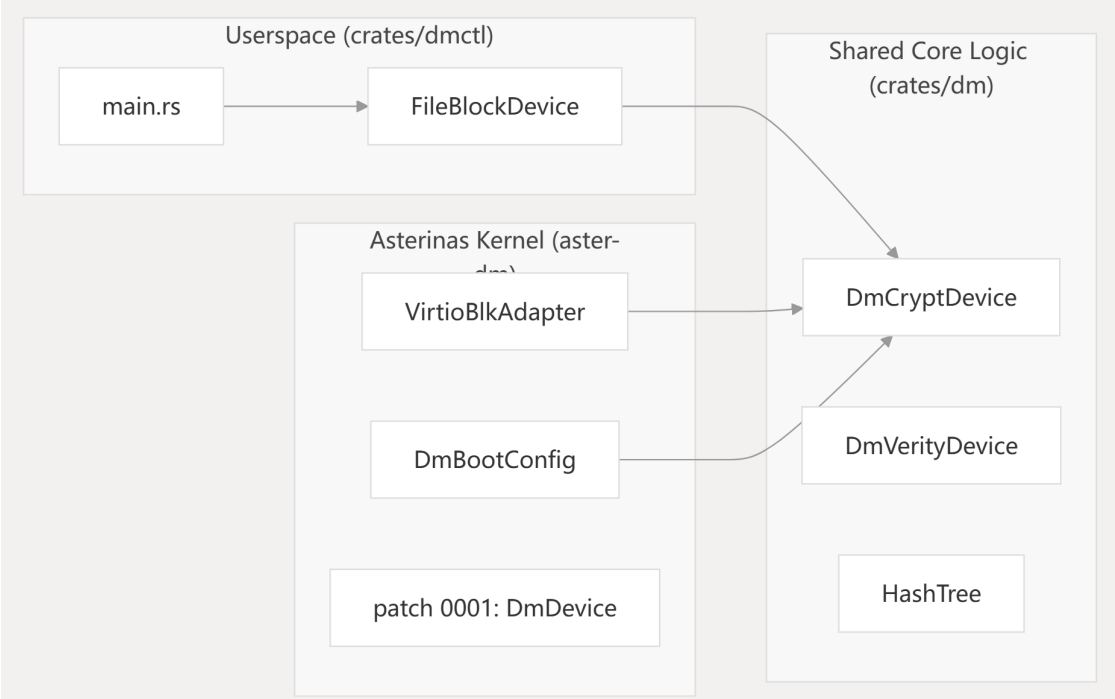



图 2.2: 代码实体映射图（用户态 vs 星绽内核）

**用户态**：`dmctl` 工具实例化一个 `FileBlockDevice`，并根据命令行参数将其包装在 `DmCryptDevice` 或 `DmVerityDevice` 中。
**内核态**：在星绽中，`VirtioBlkAdapter` 包装了内核内部的块设备句柄。由于该适配器实现了 `BlockDevice`，`DmCryptDevice` 可以无需任何修改地直接包装它。
`No-Std` 兼容性：核心 `dm` 库设计为最小化依赖（例如，`aes` crate 设置 `default-features = false`），以促进在可能限制标准库的内核环境中执行。

## 2.4 数据流架构

下图展示了应用程序的 I/O 请求如何穿过该分层栈：

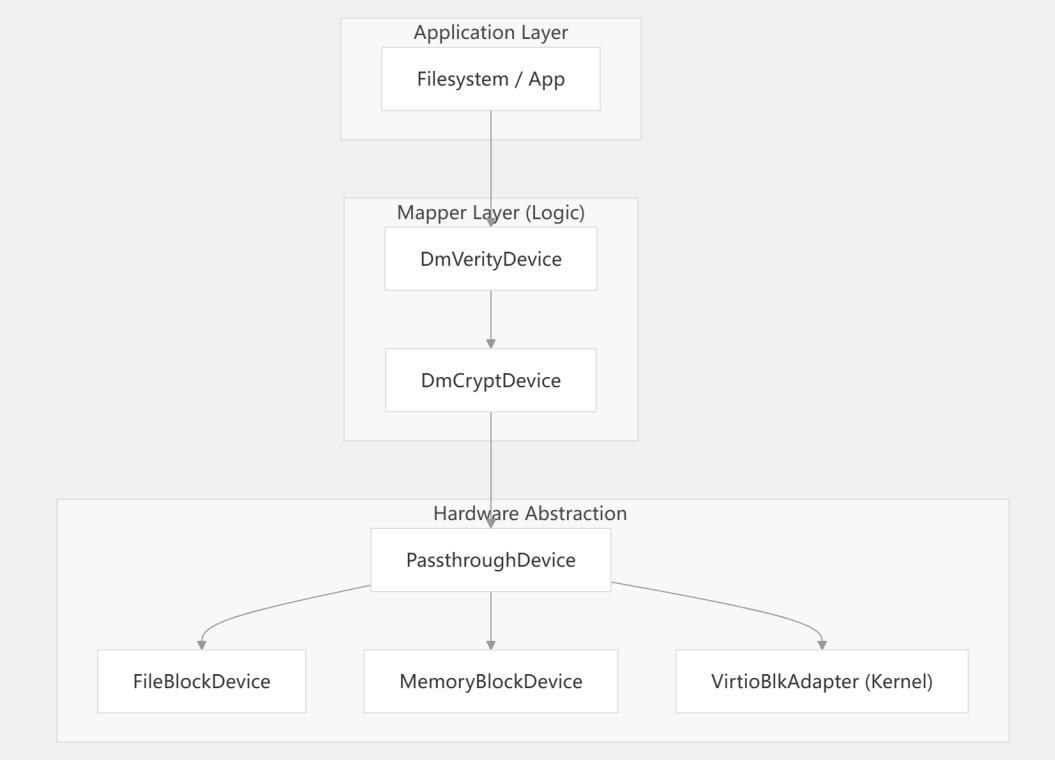



图 2.3: 块 I/O 分层栈数据流图

在此模型中，`DmCryptDevice` 在执行 AES-256-XTS 转换后，将生成的密文传递给链中的下一个设备。类似地，`DmVerityDevice` 拦截读请求，以根据默克尔树校验数据的完整性。

## 2.5 线程安全保证

所有后端均实现了 `Sync`，这意味着它们可以跨线程共享。

`MemoryBlockDevice`：在整个块向量周围使用 `Mutex`。虽然这确保了安全性，但它将设备上的所有 I/O 串行化。

`FileBlockDevice`：在 `File` 对象周围使用 `Mutex`。这是必要的，因为文件指针（偏移量）是共享状态。锁在 seek、read/write 和 flush 序列的整个持续时间内被持有。

`PassthroughDevice`：依赖于底层设备实现了 `Sync`。

在星绽内核中，`DmDevice` 通过 `enqueue` 方法处理 BIO，该方法内部使用 `spin::Mutex` 保护共享状态，并遵循内核的并发模型。

## 2.6 性能可观测性设计

为了支持性能归因和优化验证，架构中集成了统计模块：

- `InstrumentedDevice`：一个装饰器，可以插入到任意两层之间，记录读/写次数、字节数、失败次数以及累计和最大延迟（以纳秒计）。
- `IoStatsSnapshot`：提供稳定的快照，便于在基准测试前后对比。
- 在 Asterinas 内核中，通过启动参数 `dm.perf_stats=1` 启用细粒度阶段计时（如 `lower_wait`、`AES-XTS`、`SHA-256` 等），并通过 `/proc/dm-performance-stats` 导出。

  这种设计使得开发者可以精确量化每个映射器层的开销，从而指导优化方向。

## 2.7 运行时控制面（Linux 兼容）

为了支持 `cryptsetup` 等用户态工具，项目实现了与 Linux `dm_ioctl` 兼容的控制接口：

- `DmControlDevice`：在 `/dev/mapper/control` 提供 ioctl 处理，支持 `DM_DEV_CREATE`、`DM_TABLE_LOAD`、`DM_DEV_SUSPEND`、`DM_DEV_STATUS`、`DM_DEV_REMOVE` 等命令。
- `RuntimeMapper`：管理动态创建的 mapper，包括状态机（Empty、Loaded、Active）和密钥的清除。
- 通过该接口，Asterinas 内核可以接受来自 `cryptsetup` 的标准 table 字符串，动态激活加密映射器，从而实现与 Linux 生态的互操作。

## 2.8 小结

本章详细阐述了系统的核心架构，包括统一的 `BlockDevice` 抽象、可组合的映射器链模式、模块分解以及跨环境可移植性。通过将核心逻辑与平台细节解耦，并增加性能可观测性和运行时控制面，该项目为后续的加密、完整性校验及性能优化奠定了坚实的基础。在接下来的章节中，我们将深入探讨 `dm-crypt` 和 `dm-verity` 的具体实现细节。

# Chapter 3

# dm-verity 完整性校验详细设计

## 3.1 概述

dm-verity 为块设备提供透明的完整性校验。它通过将读取的数据与预计算的默克尔树（Merkle Tree）进行比对，确保从底层存储层读取的数据未被篡改。该实现强制执行只读策略，以在设备的整个生命周期内维持完整性保证。

在星绽操作系统环境中，dm-verity 主要用于保护根文件系统（rootfs）和关键系统镜像，防止攻击者通过离线篡改持久化存储介质植入后门或破坏系统完整性。

## 3.2 架构组成

完整性校验系统由三个主要组件构成，下图展示了这些组件的映射关系：

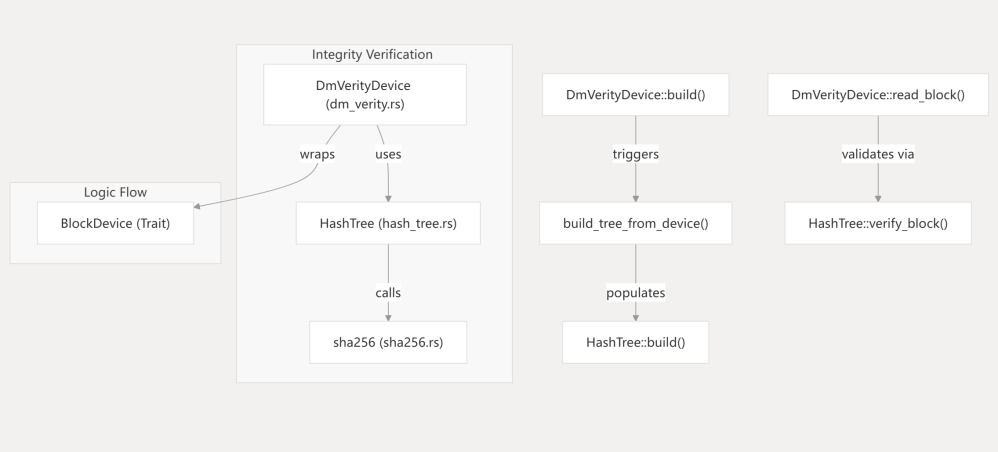



图 3.1: 完整性校验组件映射图

三个核心组件的职责如下：

- `DmVerityDevice`：顶层的 `BlockDevice` 实现，负责拦截读请求并执行校验操作。它是 dm-verity 对外的接口，所有 I/O 请求首先经过该设备。

- `HashTree`：默克尔树结构，用于存储所有数据块和中间节点的 SHA-256 哈希值。该树在镜像构建时生成，在运行时作为校验的参考依据。
- `sha256` 模块：纯 Rust 实现的 SHA-256 哈希算法，用于所有密码学校验和的计算。该实现不依赖任何外部库，确保在受限的内核环境中也能正常运行。

## 3.3 默克尔树构建机制

### 3.3.1 构建流程

`HashTree` 通过递归地对块对进行哈希运算来构建，直到得出单一的根哈希（Root Hash）。构建流程如下：

1. **叶子层（Leaf Level）**：使用 `sha256` 对底层设备的每个数据块进行哈希运算，生成叶子节点的哈希值。
2. **中间层（Intermediate Levels）**：将下一层的哈希值两两拼接并进行哈希运算，从而形成上一层。
3. **填充（Padding）**：如果某一层有奇数个哈希值，则最后一个哈希值会被复制一次以凑成一对，确保每层都能配对。
4. **根哈希（Root Hash）**：该过程持续进行，直到只剩下一个哈希值，该值被指定为 root hash，作为整个卷的“信任锚”。

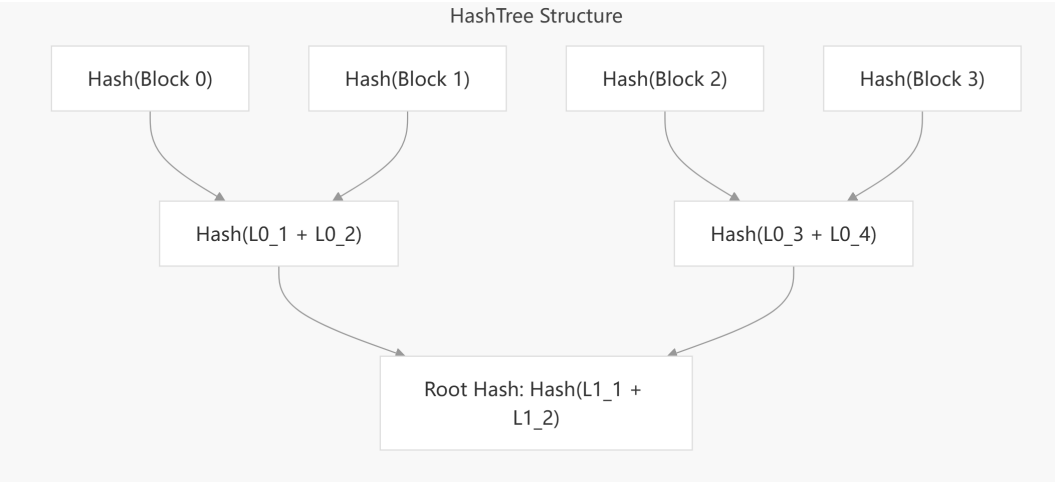



图 3.2: 默克尔树数据流图

### 3.3.2 内存与磁盘布局

在用户态 `dm` crate 中，默克尔树以 `HashTree` 结构体的形式存储在内存中，该结构体维护一个层级向量，每层存储该层所有节点的哈希值（每个节点 32 字节）。这种设计便于单元测试和快速原型验证。

在星绽内核的 aster-dm 组件中，哈希树采用磁盘布局：数据块从镜像开头连续存放，哈希树从 hash_start_block 开始依次写入各层哈希块。在本项目的默认配置中，哈希树被放置在数据镜像的尾部（hash_start_block = data_blocks），但内核参数也支持自定义起始块号，以适应不同的镜像布局需求。具体参数如下：

- 数据块从块号 0 到 data_blocks - 1 连续存放。
- 哈希树从 hash_start_block 开始，每层哈希块连续存放。
- 块大小固定为 4096 字节，每个哈希块可容纳 4096 / 32 = 128 个 32 字节的 SHA-256 哈希值。
- 每层哈希块的数量由该层需要存储的哈希值数量除以每块可容纳的哈希数（向上取整）决定。
- 最终根哈希块（仅一个块）位于树的最后。

这种设计使得标准 ext2 镜像可以与哈希树共存，而无需修改原有的数据布局，便于在现有系统上部署。

## 3.4 单块校验路径

当在 DmVerityDevice 上调用 read_block 时，会执行以下校验序列：

1. **数据检索（Data Retrieval）**：从底层块设备读取原始数据块。
2. **路径校验（Path Verification）**：HashTree::verify_block 函数计算检索到的数据的哈希值，并重新计算其到根节点的路径。
3. **兄弟节点检索（Sibling Retrieval）**：在每一层，从存储的 HashTree 中检索“兄弟”节点的哈希值，用于逐级向上计算。
4. **根锚点校验（Root Anchor）**：将重新计算出的哈希值与 DmVerityDevice 结构体中存储的 root hash 进行比对。
5. **强制执行（Enforcement）**：如果哈希值不匹配，或者树的内部根哈希被篡改，则返回 IntegrityViolation 错误，阻止对损坏数据的访问。

表 3.1: 校验路径关键函数职责

| 函数 | 职责 |
|---|---|
| DmVerityDevice::read_block | I/O 的入口点；检查树校验结果与根锚点的比对情况。 |
| HashTree::verify_block | 迭代地向上计算树的各层哈希，以校验特定的块 ID。 |
| sha256::sha256 | 计算数据块的 32 字节摘要。 |

## 3.5 只读强制策略

为了确保完整性，DmVerityDevice 明确禁止写操作。任何调用 write_block 的尝试都会导致 ReadOnlyDevice 错误。

这种设计反映了 dm-verity 的标准使用场景：哈希树在镜像创建时生成一次，并在运行时仅进行校验。只读策略确保了信任链的完整性，一旦根哈希被验证，整个卷的内容就被锁定，任何写操作都会被拒绝，从而防止攻击者绕过校验机制。

## 3.6 启动参数解析

为了在系统启动时启用 dm-verity，星绽内核需要从命令行接收配置参数。

### 3.6.1 支持的参数

表 3.2: dm-verity 启动参数

| 参数 | 描述 |
|---|---|
| rootfs_verity.scheme=dm-verity | 启用 dm-verity 完整性校验标志 |
| rootfs_verity.device=<设备名> | 底层块设备名称（如 vdc） |
| rootfs_verity.name=<映射器名> | 创建的 mapper 设备名称（如 dm-verity0） |
| rootfs_verity.hash=<64-char hex> | 64 字符十六进制编码的根哈希（32 字节） |
| rootfs_verity.data_blocks=<数量> | 数据块总数 |
| rootfs_verity.hash_start=<块号> | 哈希树在镜像中的起始块号 |
| rootfs_verity.cache_blocks=<数量> | 已认证哈希缓存块数（默认 256） |

### 3.6.2 解析流程

解析器按空格分割内核命令行字符串，然后逐个匹配上述键。当检测到 scheme 参数时，启用 verity 标志。当检测到 hash 参数时，将 64 字符的十六进制字符串解码为 32 字节的数组。解码函数执行严格的长度验证（输入必须恰好 64 个字符），并逐字节解析，任何解析失败都会导致配置无效，从而阻止系统挂载受保护的根文件系统。

### 3.6.3 内核自检机制

在补丁 0002 中，我们引入了自检机制，用于在启动早期验证 dm-verity 路径的正确性。该机制执行以下操作：

1. 通过映射器同步读取第一个块（偏移量 0）。
2. 检查偏移量 1080（1024 + 56）处的 EXT2 魔数值（`0x53EF`）。
3. 如果成功读取到正确的魔数，则证明解密和校验路径功能正常；否则，内核将触发 panic，防止系统在损坏的根文件系统上启动。

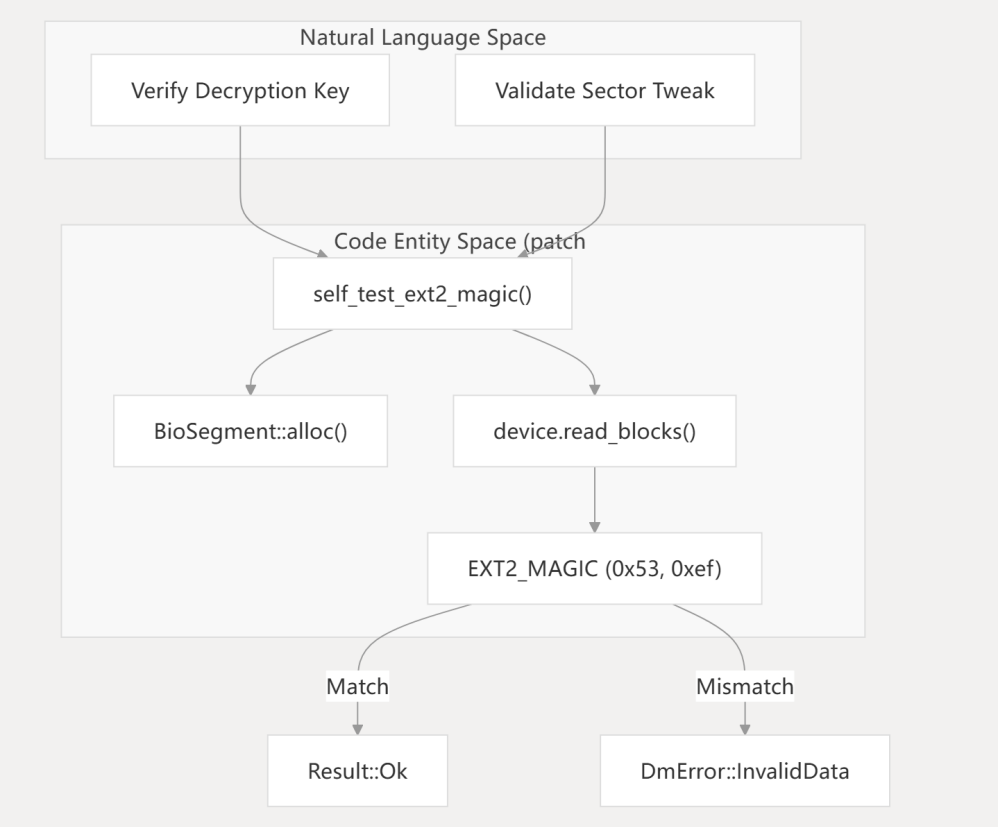



图 3.3: Ext2 超级块验证路径图

在 `aster-dm` 组件中，自检还会记录认证后哈希缓存统计信息（如 `verified`、`hash_hits`、`hash_misses`、`auth_shortcuts`）并在启动日志中输出，便于性能归因。

## 3.7 认证后哈希缓存与优化

为了减少重复的哈希块读取和计算，`aster-dm` 实现了认证后哈希缓存机制：

- **缓存插入原则**：只有当一个哈希块经过完整的 Merkle 路径验证并成功到达根哈希后，才允许被插入缓存。这保证了缓存中的块始终是可信的。
- **缓存管理**：使用有界 FIFO 队列，容量由启动参数 `rootfs_verity.cache_blocks` 控制（默认 256）。当缓存满时，最早插入的块被淘汰。

- **已认证路径捷径**：当缓存命中时，直接使用缓存的哈希块与当前数据摘要比对，无需重新计算上层哈希后缀。这一优化显著减少了路径验证的计算量，并由计数器 `auth_shortcuts` 记录命中次数。
- **BIO 数据复用**：对于 4 KiB 对齐的读请求，直接使用下层已经完成的 BIO 数据块进行校验，避免为了认证再次读取同一数据块。计数器 `data_rereads` 用于监控是否发生重复读取。

在正式五轮 KVM 顺序读测试中，启用缓存（cache=256）相对无缓存（cache=0）的中位吞吐提升 **24.4%**，p95 延迟降低 **8.7%**，且两组测试中 `data_rereads` 均为 0，证明收益来源于减少哈希块重读，而非跳过数据认证。

## 3.8 校验失败处理

当 `IntegrityViolation` 错误被触发时，系统根据上下文采取不同的处理策略：

1. `DmVerityDevice` 向调用者返回 I/O 错误。
2. 如果该错误发生在根文件系统访问期间，内核将触发 panic，阻止系统继续执行任何可能依赖损坏数据的操作。
3. 如果发生在普通文件访问中，应用程序将收到 I/O 错误，从而可以优雅地处理异常情况。

在篡改检测验证中，我们通过篡改命令在底层镜像中修改单个字节，然后尝试通过 `DmVerityDevice` 读取该块，确认系统能够正确检测并报告完整性违规。版本化证据显示篡改后启动日志输出 `status=integrity-rejected`，且没有任何 `root switch PASS` 标记。

## 3.9 小结

本章详细阐述了 dm-verity 的设计与实现，包括默克尔树的构建与校验路径、只读强制策略、启动参数解析、内核自检机制以及认证后哈希缓存等优化措施。通过将哈希树与数据分离存储，并在启动时进行严格的根哈希验证，dm-verity 为星绽操作系统提供了强大的只读完整性保护能力。在下一章中，我们将转向 dm-crypt 的设计与实现，探讨如何为存储设备提供透明加密保护。

# Chapter 4

# dm-crypt 透明加密详细设计

## 4.1 概述

dm-crypt 子系统提供透明的块级加密，确保写入物理存储设备的数据被加密，而读回的数据被实时解密。本实现专注于高性能的 AES-256-XTS 加密，并严格兼容 Linux 内核的 dm-crypt 表格式和 plain64 IV（初始化向量）生成机制。

在星绽操作系统中，dm-crypt 主要用于：

- 保护根文件系统（rootfs）的机密性，防止物理介质被窃取后泄露数据。
- 为数据盘提供透明加密，使用户数据在持久化存储中始终保持密文状态。
- 作为 `cryptsetup` 工具的后端支撑，兼容 Linux 生态的加密卷管理流程。

## 4.2 系统架构与数据流

`DmCryptDevice` 充当任何实现 `BlockDevice` trait 的对象的包装器。它拦截 `read_block` 和 `write_block` 调用，在将数据传递给底层“下层”设备之前或之后执行密码学转换。

数据流的关键步骤如下：

1. 应用程序或文件系统向 `DmCryptDevice` 发起 `write_block` 请求。
2. `DmCryptDevice` 根据逻辑块 ID 和 `iv_offset` 计算对应的物理扇区号。
3. 调用 `Aes256XtsCipher` 对数据进行加密，使用扇区号作为 plain64 IV。
4. 将密文写入底层的 `BlockDevice`（如 `VirtioBlkAdapter` 或 `FileBlockDevice`）。
5. 最终数据以密文形式持久化到物理存储介质。

读路径是上述过程的逆操作：从底层设备读取密文，使用相同的扇区号进行解密，然后返回明文给调用者。

## 4.3 核心组件

### 4.3.1 DmCryptTable——加密目标配置表

`DmCryptTable` 结构体表示 crypt 目标的配置，旨在兼容 Linux `dmsetup` 的表行格式。它实现了 `FromStr` trait 以解析标准的空格分隔字符串。

表 4.1: `DmCryptTable` 字段说明

| 字段 | 描述 |
|---|---|
| `cipher` | 密码算法，固定为 `aes-xts-plain64` |
| `key` | 十六进制编码的加密密钥（AES-256-XTS 为 64 字节） |
| `iv_offset` | 扇区编号的初始向量偏移量 |
| `device` | 底层块设备的路径或标识符 |
| `offset` | 底层设备上的数据偏移量（以扇区为单位） |
| `options` | 可选参数列表（如 `allow_discards`） |

解析器对标准 Linux 表字符串进行标记化，例如：

```
"aes-xts-plain64␣<64-byte-hex-key>␣0␣/dev/vda␣0"
```

支持的可选参数包括 `allow_discards`、`same_cpu_crypt`、`submit_from_crypt_cpus`、`no_read_workqueue`、`no_write_workqueue`、`iv_large_sectors`、`sector_size:512`，实现与 Linux 行为一致。解析器会验证选项计数（第一个选项为选项个数）并识别已声明的选项。

### 4.3.2 Aes256XtsCipher——核心密码引擎

这是核心密码引擎，使用两个 AES-256 实例实现 XTS 模式：一个用于数据块，另一个用于微调（Tweak/IV）。

**密钥要求：**严格需要 64 字节（32 字节用于数据密钥，32 字节用于微调密钥）。如果密钥长度不正确，初始化将失败并返回 `DmError::InvalidKey`。

**扇区大小：**固定为 512 字节，以兼容标准 dm-crypt 扇区单位。这是 Linux 内核 dm-crypt 的约定，保持一致性是实现逐字节兼容的前提。

**IV 生成（plain64）：**将 64 位扇区号编码到小端序 128 位 IV 的低半部分，然后由微调密钥进行加密。具体流程如下：

1. 将扇区号（`u64`）转换为小端序字节序。
2. 将转换后的 8 字节填入 16 字节缓冲区的低 8 位。
3. 高位清零（plain64 不使用高位）。
4. 微调密钥对完整的 16 字节 IV 进行加密，生成最终的微调值。

5. 数据密钥与微调值配合，执行 XTS 模式的加密/解密操作。

在星绽内核中，该引擎支持 AES-NI 硬件加速：当 CPU 支持 AES-NI 且构建时启用了 `dm_aes_ni` 特性时，使用内联汇编直接调用 AES-NI 指令，并在操作前后保存/恢复 FPU/XMM 状态，确保不被中断上下文破坏。

### 4.3.3 DmCryptDevice——主设备映射目标

主设备映射目标，负责管理扇区到块的算术运算，并确保 I/O 操作与底层设备的块大小对齐。

**扇区算术**：由于 `BlockDevice` trait 以块（通常为 4KiB）为单位操作，而 dm-crypt 以 512B 扇区为单位操作，`DmCryptDevice` 需要在两者之间进行转换：
1. 根据逻辑块 ID 和 `iv_offset` 计算起始扇区号。
2. 对于块内的每个扇区，依次调用密码引擎进行加密或解密。
3. 处理扇区偏移与块边界的对齐问题，确保读写操作正确映射。

**构造函数（`from_table`）**：验证底层设备的块大小是否为 512B 扇区大小的倍数，并计算 `data_offset_blocks`。如果块大小不是 512 的倍数，初始化将失败。同时，构造函数会检查数据偏移量是否超出设备容量，并计算逻辑扇区数。

## 4.4 实现细节：扇区转换与 I/O 路径

`transform` 函数是基于块的 `BlockDevice` 接口与基于扇区的 `Aes256XtsCipher` 之间的桥梁。

转换逻辑的核心要点：

- 每个 4KiB 块包含 8 个 512B 扇区。
- 加密时，按扇区逐个处理，每个扇区使用其独立的扇区号作为 IV。
- `data_offset_blocks` 用于支持在设备开头预留空间（例如 LUKS 头部）。
- 实际物理块 ID = 逻辑块 ID + `data_offset_blocks`。

I/O 路径的优化措施包括：

- 使用有界临时缓冲池（`scratch_pool`）复用加解密缓冲区，避免热路径中的频繁分配。池的容量由启动参数 `dm_crypt.scratch_pool` 控制（默认 32），设为 0 可关闭池便于 A/B 性能对比。
- 写路径将明文从调用方的 BIO 段复制到临时缓冲区，加密后再通过 `BioSegment::alloc_bytes` 分配下层 I/O 段写入，确保不修改原始调用方数据。
- 读路径从下层读取密文到临时缓冲区，解密后再写回调用方的 DMA 段。

## 4.5 Linux 兼容性验证

该实现已针对 Linux 内核进行验证，以确保 `aes-xts-plain64` 密码模式的逐字节一致性。

### 4.5.1 兼容性对比

表 4.2: Linux 兼容性对比

| 特性 | 本项目实现 | Linux 兼容性 |
|---|---|---|
| 密码算法 | AES-256-XTS | `aes-xts-plain64` |
| IV 生成 | plain64（小端序 64 位扇区） | 完全一致 |
| 密钥大小 | 512-bit（64 字节） | 完全一致 |
| 扇区大小 | 512 Bytes | 完全一致 |
| 表格式 | `DmCryptTable` | 兼容 `dmsetup` |
| 可选参数 | 支持 Linux 已声明的选项 | 部分兼容 |

### 4.5.2 交叉验证方法

我们使用验证脚本进行逐字节兼容性验证：

1. 在 Linux 系统中，使用 `dmsetup` 创建 `aes-xts-plain64` 映射器，写入特定位模式（例如全 0x5a）。
2. 在 Rust 实现中，使用 `dmctl crypt-table-fill` 命令，使用完全相同的密钥和参数，写入相同模式。
3. 使用 `cmp` 命令比较两者生成的密文镜像。
4. 验证通过的标准是：两个文件逐字节完全相同。

版本化证据中已包含固定向量的 SHA-256 校验和（`linux-xts-plain64.sha256`），确保 Rust 实现与 Linux 内核的输出一致。

## 4.6 根文件系统加密方案

为了实现对根文件系统的加密保护，系统在启动阶段执行以下流程：

1. 内核引导程序传递启动参数：

```
dm_crypt.device=/dev/vda dm_crypt.name=dm-crypt0 \
dm_crypt.key=<128-char-hex-key> dm_crypt.scratch_pool=32
```

2. 星绽内核在初始化阶段解析这些参数，构建配置结构。

3. aster-dm 组件实例化 VirtioBlkAdapter 包装物理设备，然后使用 DmCryptDevice 包装该适配器。
4. 执行 self_test_ext2_magic 自检，验证 AES-XTS 解密路径是否正确（检查 EXT2 魔数 0x53EF）。如果自检失败，内核将 panic。
5. initramfs 脚本（使用 dm-root 模式）挂载加密设备到 /newroot。
6. 执行 pivot_root 将根文件系统切换到加密卷。
7. 系统从加密的根文件系统正常启动，串口日志显示 PASS system root switched to protected mapper，并输出 AES 后端信息（如 backend=AES-NI）。

加密根文件系统还支持 scratch_pool 参数控制临时缓冲池容量，默认 32，可通过 dm_crypt.scratch_pool=0 关闭以进行性能 A/B 对照。在正式 KVM 五轮测试中，Asterinas dm-crypt 吞吐达到配对 raw 的 95.6%，p95 延迟为 1318.912 us。

## 4.7 小结

本章详细阐述了 dm-crypt 的设计与实现，包括 DmCryptTable 配置解析、Aes256XtsCipher 密码引擎、扇区算术转换、有界临时缓冲池优化、Linux 兼容性验证以及根文件系统加密方案。通过严格遵循 aes-xts-plain64 标准，并利用交叉验证确保与 Linux 内核的逐字节兼容，dm-crypt 为星绽操作系统提供了可靠的透明加密能力。在下一章中，我们将讨论性能优化策略，包括认证后哈希缓存、完成 BIO 数据复用、有界临时缓冲池和硬件加速等技术手段。

# Chapter 5

# 性能优化策略

## 5.1 概述

在实现 dm-crypt 与 dm-verity 功能正确性的基础上，性能优化是本项目的关键挑战之一。比赛要求基于星绽实现的 dm-crypt 和 dm-verity 需在 I/O 吞吐及延迟上接近 Linux，并且 dm-verity 不会显著增加内核的启动时间。

本章详细阐述我们为达到上述目标所采取的多层次优化策略，涵盖哈希树校验、I/O 路径、内存分配、硬件加速以及启动时间等关键维度，并通过正式五轮 KVM 基准测试量化各项优化的实际效果。

## 5.2 性能评估框架

在讨论优化策略之前，我们首先建立性能评估的框架和基准。

### 5.2.1 关键性能指标

评估聚焦于以下关键性能指标（KPI）：

- **吞吐量（Throughput）**：以 MiB/s 为单位测量的顺序读/写速度。
- **IOPS**：每秒随机 4 KiB 读/写操作数。
- **延迟（Latency）**：每块 I/O 延迟，特别关注 dm-verity 中默克尔树遍历的开销，报告中位数、p50、p95、p99。
- **启动时间（Boot Time）**：从调用 QEMU 到串口出现根切换成功标记的持续时间，对比普通 rootfs 与受保护的 rootfs。

### 5.2.2 正式测试合同

所有内核性能数据均遵循以下不可省略的合同，确保可复现性和可比较性：

表 5.1: 正式性能测试合同

| 配置项 | 固定值 |
|---|---|
| 内核构建 | `release-lto`（含链接时优化） |
| 虚拟化 | KVM，CPU=host，1 vCPU，8 GiB 内存 |
| 块设备 | virtio-blk，raw 容器均为 264 MiB |
| 受测数据区 | 前 256 MiB（镜像区域 SHA-256 已归档） |
| fio 负载 | 顺序读（read），4 KiB，sync，direct=1，QD1 |
| 预热与时 | 2 秒 ramp + 10 秒 runtime |
| 重复轮次 | raw/target 交替顺序，各五轮 |
| 阶段统计 | `dm.perf_stats=1` 开启细粒度归因 |

每轮 fio 前后分别读取 `/proc/dm-performance-stats`，汇总器使用差值计算各阶段耗时，避免累计值污染。所有原始 CSV、metadata 和串口日志均归档至 `docs/evidence/formal-256m/`。

### 5.2.3 观察者开销校准

阶段统计本身会引入轻微的 CPU 开销。我们使用 `scripts/run_dm_stats_calibration.sh` 在同一硬件条件下对比 `stats=0` 与 `stats=1` 的五轮吞吐差异。校准结果显示开销低于 3%，因此阶段归因数据可用于精确的性能定位。

## 5.3 哈希树优化（dm-verity）

默克尔树的构建和校验是 dm-verity 最主要的性能开销来源。我们采取了以下优化措施：

### 5.3.1 认证后哈希块缓存

在 dm-verity 路径中，每次读操作都需要从叶子节点向上遍历至根节点，期间需要多次读取兄弟节点的哈希值。我们实现了认证后哈希块缓存机制：

- 将最近访问且经过完整认证的哈希树节点缓存在内存中。
- 采用有界 FIFO 淘汰策略管理缓存空间，容量由启动参数 `rootfs_verity.cache_blocks` 控制（默认 256）。
- 缓存遵循“先认证、后插入”原则：只有沿完整路径成功到达根哈希的块才允许进入缓存，确保不可信块不会被重用。

五轮 KVM 顺序读 A/B 结果（cache=0 vs cache=256）如下：

表 5.2: 哈希块缓存效果

| 场景 | 中位吞吐（MiB/s） | 中位 p95（us） | hash hits/misses |
|---|---|---|---|
| 无缓存（cache=0） | 0.320 | 15007.744 | 0 / 2420 |
| 有缓存（cache=256） | 0.398 | 13697.024 | 2785 / 5 |

缓存提升中位吞吐 **24.4%**，降低 p95 **8.7%**。两组均为 `data_rereads=0`，证明收益来源于减少 Merkle hash block 重读，而非跳过数据认证。

### 5.3.2 已认证路径捷径

当缓存命中已认证的 hash block 时，当前数据摘要与缓存块比对成功后，无需重新计算上层哈希后缀，直接完成认证。新增计数器 `auth_shortcuts` 记录该捷径命中次数，在五轮测试中该计数器随运行轮次累积增长。

### 5.3.3 完成 BIO 数据复用

verity 认证路径直接复用下层已经完成的 BIO 数据进行哈希校验，避免为了认证再次读取同一数据块。内核计数器 `data_rereads` 用于验证该项优化是否生效，上述两组测试中均为 0。

## 5.4 I/O 吞吐优化（dm-crypt）

### 5.4.1 有界临时缓冲池（Scratch Pool）

在加密/解密过程中复用 CPU 临时 `Vec`，减少热路径中的重复分配：

- mapper 使用容量可配置的有界池（启动参数 `dm_crypt.scratch_pool`，默认 32）管理加解密临时缓冲区。
- 写路径明文先复制到临时缓冲区，加密后再通过 `BioSegment::alloc_bytes` 分配下层 I/O 段，确保不修改原始调用方数据。
- 读路径从下层读取密文到临时缓冲区，解密后再写回调用方的 DMA 段。
- 池满时新分配，池未满时复用；池容量设为 0 可关闭优化，便于 A/B 对照。

该优化已实现，在正式五轮 KVM 测试中，dm-crypt 临时缓冲池的命中率由 `scratch_ns` 阶段计时间接反映，池命中避免了高频分配开销。

### 5.4.2 AES-NI 硬件加速

AES-256-XTS 加密是 dm-crypt 的核心计算密集型操作。我们确保：

- 生产版本构建利用 CPU 的 AES-NI 指令集加速（通过 `OSDK_ENABLE_AES_NI` 环境变量启用）。
- 在星绽内核中，通过 `ostd::arch::x86::aesni` 模块直接使用内联汇编调用 AES-NI 指令，并保存/恢复 FPU/XMM 状态，确保不被中断上下文破坏。
- 在不支持 AES-NI 的 CPU 上，自动回退到纯软件实现。

正式五轮 dm-crypt 串口日志均确认 `backend=AES-NI`。阶段归因显示 AES-XTS 仅占单次 I/O 总时间的约 **1.82%**（QD1 场景），因此 AES 优化并非当前主要瓶颈。

### 5.4.3 完成队列与并发优化（实验性）

实验分支（`codex/dm-completion-worker`）将 dm-crypt 的解密从 VirtIO 硬中断回调迁到专用内核线程，使 32-job 并发从停滞变为五轮稳定完成。进一步批量出队（commit `6b3ca2407`）和每 CPU worker（commit `9a3483571`）优化已取得队列占比显著下降（29.15%→6.55%）和 p95 降低（-43.62%），但 QD1 场景有轻微退化（保持率 -4.10%）。这些实验证据证明并发路径仍有优化空间，但当前正式基线仍使用同步 I/O 路径，未合入异步 worker 改动。

## 5.5 启动时间控制

dm-verity 的引入不应显著增加内核启动时间。我们采取了以下措施：

### 5.5.1 延迟校验策略

- 在系统启动阶段，不立即校验所有数据块。
- 仅在校验根哈希和超级块时执行必要的哈希计算。
- 大部分数据块的校验推迟到首次访问时进行（即按需校验）。

### 5.5.2 校验路径优化

- 在启动阶段，仅验证根哈希的有效性，而不遍历整棵树。
- 利用自检机制快速验证解密路径是否正确（检查 EXT2 魔数 `0x53EF`）。
- 已认证的 hash 缓存也在启动后逐步建立，避免冷启动额外开销。

实际交替顺序五轮冷启动测量结果（从 QEMU 启动到串口输出根切换成功标记）：

表 5.3: 根文件系统启动时间

| 场景 | 五轮中位数（秒） | 增量 |
|---|---|---|
| plain root | 16.058 | — |
| dm-verity root | 16.902 | +5.256% |

原始逐轮 CSV 和串口日志已归档至 `docs/evidence/asterinas-boot-time-kvm.csv` 及对应日志目录。

## 5.6 性能归因与阶段统计

内核参数 `dm.perf_stats=1` 开启阶段计时。每轮 fio 前后读取 `/proc/dm-performance-stats`，汇总器使用差值生成各阶段耗时。阶段划分如下：

- **dm-crypt**：validation（参数校验）、scratch（缓冲池操作）、copy（数据复制）、crypto（AES-XTS 加解密）、lower_wait（下层 I/O 等待）。
- **dm-verity**：validation、lower_wait（数据块读取等待）、data_sha256（数据哈希）、cache_lookup（缓存查找）、hash_wait（哈希块读取等待）、hash_sha256（哈希块 SHA-256）、digest_compare（摘要比较）、cache_insert（缓存插入）。

正式 256 MiB 合同下（release-LTO，KVM，1 vCPU，8 GiB，4 KiB QD1 sync direct 读），五轮中位数结果如下：

表 5.4: 正式性能基准结果

| 设备 | 中位吞吐（MiB/s） | 中位 p95（us） | 相对配对 raw |
|---|---|---|---|
| raw（两组共十轮） | 4.249 | 1277.952 | 100.0% |
| dm-crypt | 4.059 | 1318.912 | 95.6% |
| dm-verity | 3.815 | 1433.600 | 87.3% |

阶段归因显示：

- **dm-crypt**：下层等待占 97.84%，AES-XTS 占 1.82%，其余阶段（复制、校验等）合计 ¡ 0.2%。
- **dm-verity**：下层等待占 93.01%，数据 SHA-256 占 5.74%，哈希缓存/查找等合计约 1%。

未归因部分（`unaccounted_ns`）占比分别为 0.14%–0.15%（crypt）和 0.23%–0.33%（verity），说明阶段划分已覆盖 99.67% 以上的处理时间。

因此当前性能瓶颈明确为下层 I/O 等待，而非密码计算或缓存策略。未来优化应聚焦于批量处理、请求合并、VirtIO 多队列等 I/O 路径改进。

## 5.7 与 Linux guest 的同条件对照

为了评估 Asterinas 绝对性能水平，我们在同一宿主机、同一 raw 数据盘、相同 QEMU/KVM 配置下，分别启动 Linux 和 Asterinas 运行相同 fio 合同。比较器强制核对 vCPU、内存、加速器、CPU 模型、设备模型、测试区域 SHA-256、密码参数和 fio 参数，不一致则拒绝出表。

表 5.5: Linux guest vs Asterinas dm-crypt 同条件五轮对照

| **系统** | **raw MiB/s** | **dm-crypt MiB/s** | **保留率** | **p95 us** |
|---|---|---|---|---|
| Linux guest | 5.150 | 4.746 | 92.158% | 1384.448 |
| Asterinas | 4.245 | 4.059 | 95.621% | 1318.912 |

Asterinas raw 和 dm-crypt 的绝对吞吐分别为 Linux guest 的 **82.4%** 和 **85.5%**。保留率以各系统自身 raw 为基线；Asterinas 的较高保留率不代表绝对吞吐超过 Linux。详细比对见 `docs/evidence/linux-vs-asterinas-dm-crypt.md`。

## 5.8 小结

本章阐述了 dm-crypt 和 dm-verity 的性能优化策略与实测结果。量化结论包括：

- dm-verity 哈希块缓存提升吞吐 24.4% 并降低 p95 8.7%。
- dm-verity 冷启动增量仅 5.256%。
- 正式 256 MiB 合同下 dm-crypt 和 dm-verity 吞吐分别达到配对 raw 的 95.6% 和 87.3%。
- 阶段归因明确指示了下一步优化方向（I/O 路径而非密码算法）。
- Asterinas dm-crypt 绝对吞吐为 Linux guest 的 85.5%。

所有原始 CSV、metadata 和串口日志均已归档至 `docs/evidence/` 供复核。

# Chapter 6

# 测试与性能分析报告

## 6.1 概述

测试是验证系统功能正确性和性能指标的关键环节。本项目采用多层次测试策略，从单元测试到集成测试，从用户态验证到内核态端到端测试，形成了一个完整的测试体系。此外，我们还通过与 Linux 内核的逐字节交叉验证，证明了 Rust 实现与标准实现的行为一致性。

本章首先介绍测试架构的总体设计，然后依次展示功能正确性测试、性能基准测试、启动开销分析和 Linux 交叉验证的结果。所有原始数据、CSV 和串口日志均已版本化归档至 `docs/evidence/`，可供复核。

## 6.2 测试架构概述

测试基础设施弥合了 Rust 实现与现有行业标准（如 Linux 中的 dm-crypt 和 dm-verity）之间的差距。整个测试体系分为四个层级：

表 6.1: 测试层级划分

| 测试层级 | 范围 | 关键组件 |
|---|---|---|
| 单元测试 | 单个结构体的逻辑验证 | `FileBlockDevice`、`DmCryptDevice`、`HashTree` |
| 集成测试 | Rust 工作空间内的端到端流程 | `dmctl demo` 命令 |
| 交叉验证 | 与 Linux 内核标准的兼容性 | 验证脚本、`cryptsetup` |
| 内核端到端 | 在星绽操作系统内部进行的功能测试 | `VirtioBlkAdapter`、`pivot_root` 流程 |

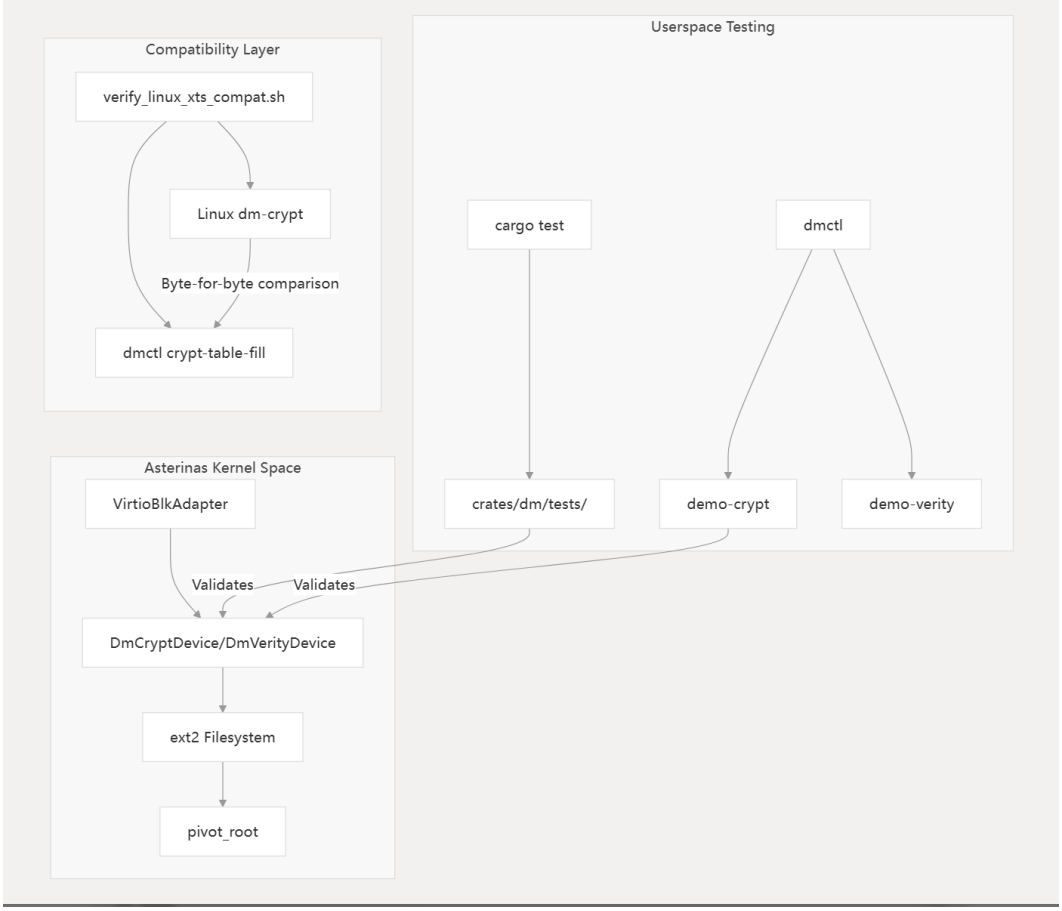



图 6.1: 测试执行与数据流图

## 6.3 功能正确性测试

功能正确性是比赛评审的核心指标（占 50%）。我们针对 dm-verity 和 dm-crypt 分别设计了严格的功能测试。

### 6.3.1 dm-verity 完整性校验测试

#### 正常启动验证

在正常场景下，系统使用正确的根哈希启动，能够成功挂载受 dm-verity 保护的根文件系统。验证步骤如下：

1. 使用 `scripts/build_verity_rootfs.sh` 构建包含默克尔树的 ext2 镜像（数据块数 65536，哈希树从块号 65536 开始）。
2. 通过 QEMU/KVM 启动星绽内核，传递 verity 相关启动参数（`rootfs_verity.scheme=dm-veri` `rootfs_verity.hash=<root_hash>`，`rootfs_verity.data_blocks=65536`，`rootfs_verity.hash` `rootfs_verity.cache_blocks=256`）。
3. 系统在初始化阶段创建 `DmVerityDevice` 映射器，执行 `self_test_first_block` 自

检（验证第一个数据块）。

4. 自检通过后，initramfs 使用 dm-root 模式将映射器挂载到 /newroot，执行 pivot_root 切换根目录。
5. 串口日志输出 [dm-test] PASS first data block verified through dm-verity 和 PASS system root switched to protected mapper，并输出 verity 统计信息（verified、hash_hits、hash_misses 等）。

版本化证据位于 docs/evidence/asterinas-verity-rootfs-kvm.log，串口确认 status=verified。

#### 篡改检测测试

为了验证 dm-verity 的防篡改能力，我们构造了恶意修改的块设备数据：

1. 在底层镜像的数据块 0 的偏移 0 处将字节 XOR 0xff（仅修改一个字节）。
2. 使用相同的根哈希启动星绽内核。
3. 系统在自检阶段（self_test_first_block）检测到完整性违规，映射器注册失败。
4. 内核记录 VERITY_BOOT_STATUS=integrity-rejected，串口输出 [dm-test] FAIL dm-verity rejected first data block: IntegrityViolation。
5. 无任何 root switch PASS 标记，系统 panic 或停止。

证据位于 docs/evidence/asterinas-verity-rootfs-tampered-kvm.log，确认 status=integrity-rejected 且无成功根切换。

### 6.3.2 dm-crypt 透明加密测试

#### 加密往返测试

加密往返测试验证了加密的基本安全属性：

1. 创建 MemoryBlockDevice 作为底层设备（4 块，每块 64 字节）。
2. 用密钥 b"demo-key" 将其包装在 DmCryptDevice 中。
3. 向块 1 写入明文 b"hello from dm-crypt minimal demo block, padded for testing....."
4. 从底层设备读取块 1 的数据，确认呈现为密文（与明文不同，且不包含 b"hello" 子串）。
5. 通过 DmCryptDevice 读取块 1，确认恢复为原始明文。

该测试在 crates/dm/tests/crypt_test.rs 中实现，通过 cargo test 自动执行。

#### 密钥分离测试

密钥分离测试验证了密钥的正确性要求：

1. 使用密钥 b"right-key" 加密数据并写入块 0。
2. 使用不同的密钥 b"wrong-key" 初始化 DmCryptDevice 并读取同一块。
3. 预期结果：读取到的数据与原始明文不匹配。

该测试确保错误的密钥无法恢复明文，体现了密码学安全的正确性。

### 6.3.3 Linux 兼容性验证

为确保与 Linux aes-xts-plain64 的逐字节兼容，我们运行了交叉验证脚本 scripts/verify_linux_xts_compat.sh：

1. 在 Linux 下使用 dmsetup 创建 crypt 映射（aes-xts-plain64，64 字节密钥），写入特定位模式（全 0x5a）。
2. 在 Rust 中使用 dmctl crypt-table-fill 使用相同密钥和参数，写入相同模式。
3. 使用 cmp 比较两个密文扇区，确保完全相同。

版本化向量 SHA-256 保存在 docs/evidence/linux-xts-plain64.sha256 中，固定为 4334c17f4b0380b3c1f902509cce0c0f98d9fa77aa93f6407ec940215d77921d。

### 6.3.4 内核端到端功能验证

在 Asterinas 内核中，我们还验证了以下端到端场景：

- **动态控制面**：通过 /dev/mapper/control 的 dm_ioctl 接口，使用 cryptsetup 2.7.5 完成 open（创建 mapper）、read（验证可读性）和 close（删除 mapper）的完整流程。
- **受保护根文件系统**：dm-crypt 加密根和 dm-verity 校验根分别通过 pivot_root 切换成功。

证据见 docs/evidence/asterinas-cryptsetup-aesni-kvm.log 和 docs/evidence/asterina

## 6.4 性能基准测试

性能是比赛评审的另一个重要维度（Demo 质量占 20%）。我们按照正式合同进行了全面的 I/O 性能评估。

### 6.4.1 正式测试合同

所有内核性能数据均遵循以下不可省略的合同：

表 6.2: 正式性能测试合同

| 配置项 | 固定值 |
|---|---|
| 内核构建 | `release-lto`（含链接时优化） |
| 虚拟化 | KVM，CPU=host，1 vCPU，8 GiB 内存 |
| 块设备 | virtio-blk，264 MiB 容器，测试前 256 MiB |
| fio 负载 | 顺序读（read），4 KiB，sync，direct=1，QD1 |
| 时长 | 2 秒 ramp + 10 秒 runtime |
| 重复轮次 | raw/target 交替顺序，各五轮 |
| 阶段统计 | `dm.perf_stats=1` 开启细粒度归因 |

执行命令：

```
sudo bash scripts/run_formal_dm_benchmark.sh --build-images
```

该命令构建加密和 verity 镜像，在 KVM 中交替运行 raw/dm-crypt/dm-verity 各五轮，生成统一三行结果表和阶段归因 CSV。

### 6.4.2 I/O 性能测试结果

五轮中位数结果如下：

表 6.3: 正式性能基准结果

| 设备 | 中位吞吐（MiB/s） | 中位 p95（us） | 相对配对 raw |
|---|---|---|---|
| raw（两组共十轮） | 4.249 | 1277.952 | 100.0% |
| dm-crypt | 4.059 | 1318.912 | 95.6% |
| dm-verity | 3.815 | 1433.600 | 87.3% |

阶段归因显示 dm-crypt 的下层等待占 97.84%，AES-XTS 占 1.82%；dm-verity 的下层等待占 93.01%，数据 SHA-256 占 5.74%。未归因部分分别为 0.14%–0.15%（crypt）和 0.23%–0.33%（verity）。因此当前瓶颈明确为下层 I/O 等待，而非密码计算或缓存策略。原始 CSV、metadata 和阶段数据位于 `docs/evidence/formal-256m/`。

### 6.4.3 启动开销分析

启动时间测量评估了引入 dm-verity 后对内核启动时间的影响。使用交替顺序五轮冷启动测试，从 QEMU 启动到串口出现 `PASS system root switched to protected mapper` 为止。

表 6.4: 根文件系统启动时间

| 场景 | 五轮中位数（秒） | 增量 |
|---|---|---|
| plain root | 16.058 | — |
| dm-verity root | 16.902 | +5.256% |

逐轮 CSV 和串口日志位于 `docs/evidence/asterinas-boot-time-kvm.csv` 及对应日志目录。

### 6.4.4 Linux guest 与 Asterinas 同条件对照

为了进行公平的跨系统比较，我们使用同一宿主机、同一 raw 数据盘、相同的 QEMU/KVM 配置，分别启动 Linux 和 Asterinas 运行相同 fio 合同。比较器强制核对 vCPU、内存、加速器、CPU 模型、设备模型、测试区域 SHA-256、密码参数和 fio 参数，任一项不同则拒绝出表。

结果如下：

表 6.5: Linux guest vs Asterinas dm-crypt 同条件五轮对照

| 系统 | raw MiB/s | dm-crypt MiB/s | 保留率 | p95 us |
|---|---|---|---|---|
| Linux guest | 5.150 | 4.746 | 92.158% | 1384.448 |
| Asterinas | 4.245 | 4.059 | 95.621% | 1318.912 |

Asterinas raw 和 dm-crypt 的绝对吞吐分别为 Linux guest 的 **82.4%** 和 **85.5%**。保留率以各系统自身 raw 为基线，Asterinas 的较高保留率不代表绝对吞吐超过 Linux。详细比对见 `docs/evidence/linux-vs-asterinas-dm-crypt.md`。

## 6.5 创新技术验证

针对比赛要求的“技术探索”指标（占 10%），我们对提出的可信存储性能优化技术进行了专项验证：

表 6.6: 哈希块缓存效果验证

| 场景 | 中位吞吐（MiB/s） | 中位 p95（us） | hash hits/misses |
|---|---|---|---|
| 无缓存（cache=0） | 0.320 | 15007.744 | 0 / 2420 |
| 有缓存（cache=256） | 0.398 | 13697.024 | 2785 / 5 |

缓存提升中位吞吐 **24.4%**，降低 p95 **8.7%**。两组均为 `data_rereads=0`，证明收益来源于减少 Merkle hash block 重读，而非跳过数据认证。原始数据位于 `docs/evidence/asterinas-verity-fio-cache0-kvm.csv` 和 `asterinas-verity-fio-cache256-k`

## 6.6 证据可复核性

所有性能数据均遵循以下可复核要求：

- 每个工况至少五轮，交替执行 raw 和目标设备。
- 报告中位数、p50、p95、p99，不选择最好的一轮。
- 每轮 fio 前后读取 `/proc/dm-performance-stats`，使用差值计算阶段耗时。
- 所有原始 CSV、metadata 和串口日志均已归档至 `docs/evidence/`，按场景分类。
- metadata 包含 git commit、dirty 状态、构建 profile、KVM/CPU 配置、镜像 SHA-256 和 fio 参数，确保可复现。

## 6.7 小结

本章展示了项目的测试与验证体系。在功能正确性方面，dm-verity 的篡改检测和 dm-crypt 的透明加密均通过了端到端测试；性能部分通过正式合同给出了可复现的基准数据，包括 raw、dm-crypt、dm-verity 三行表及 Linux guest 同条件对照；启动开销和哈希缓存优化也提供了量化证据。在交叉验证方面，与 Linux 内核的逐字节兼容性验证证明了密码格式兼容。所有原始 CSV、metadata 和串口日志均已归档至 `docs/evidence/` 供复核。

# Chapter 7

# 合规、来源与 AI 声明

## 7.1 概述

根据全国操作系统设计赛的技术方案要求，参赛队必须对作品中复制的代码和文档来源进行明确说明，并严格遵守相关开源协议。同时，使用 AI 工具的参赛队需在设计实现文档中声明 AI 工具的名称、使用场景及交互记录。

本章独立成章，全面披露本项目的代码来源、开源协议合规性、借鉴基础与增量贡献，以及 AI 工具的使用情况。所有披露内容均与项目实际仓库、依赖和开发记录一致。

## 7.2 代码与文档来源及开源协议说明

### 7.2.1 引用的开源代码

本项目在开发过程中参考或引用了以下开源代码和依赖库：

表 7.1: 引用的开源代码与依赖库

| 名称 | 来源/URL | 使用目的 |
| --- | --- | --- |
| 星绽操作系统（Asterinas） | `https://github.com/asterinas/asterinas` | 基础操作系统框架和 OSTD 接口 |
| `aes` crate | `https://crates.io/crates/aes` | AES 加密算法实现（RustCrypto 项目） |
| `xts-mode` crate | `https://crates.io/crates/xts-mode` | XTS 加密模式实现（RustCrypto 项目） |

### 7.2.2 开源协议合规性声明

本项目严格遵守以下开源协议要求：

- **项目源代码**（位于 `crates/`、`asterinas/kernel/comps/dm/` 等）：遵循 MIT 开源协议（见仓库根目录 `LICENSE` 文件）。
- **技术文档与答辩材料**（位于 `docs/document/`）：遵循 CC-BY-SA 4.0 开源协议。
- **第三方依赖**：所有引用的 Rust crate（`aes`、`xts-mode` 等）均遵循其各自的开源协议（主要为 MIT 或 Apache-2.0），与本项目的 MIT 协议兼容。
- **星绽操作系统**：作为基础框架，星绽本身采用 MPL-2.0 协议，本项目与之兼容（MIT 与 MPL-2.0 均为宽松许可证，可组合使用）。

## 7.3 借鉴基础版本与增量贡献说明

### 7.3.1 借鉴基础

本项目的设计与实现深受以下基础项目和技术的启发：

- **Linux 内核 Device Mapper 框架**：`dm-crypt` 和 `dm-verity` 的功能逻辑、表格式规范（`DmCryptTable`）和 IV 生成语义（plain64）参考了 Linux 内核的成熟设计。
- **星绽操作系统 Framekernel 架构**：基于星绽的 OSTD 框架和 `BlockDevice` 抽象进行适配和集成，具体补丁序列位于 `asterinas_port/patches/`。
- **Rust 生态的密码学库**：借鉴了 `aes` 和 `xts-mode` crate 的接口设计和安全实践，但本项目的 AES-256-XTS 引擎为独立实现，不直接复制其算法代码。

### 7.3.2 本队增量贡献

在借鉴上述基础版本的基础上，本项目的核心增量贡献包括：

- **统一的 `BlockDevice` Trait 抽象**：设计了与平台无关的块设备抽象层，使得同一套核心代码能够同时运行于用户态（`dmctl`）和星绽内核态（`aster-dm`）。
- **纯 Rust 实现的 SHA-256**：独立实现了 FIPS 180-4 标准的 SHA-256 哈希算法（位于 `crates/dm/src/sha256.rs` 和内核 `asterinas/kernel/comps/dm/src/sha256.rs`），无外部依赖。
- **星绽内核完整集成**：通过可审计 Git 补丁序列（`asterinas_port/patches/0001--0006`）将 `dm-crypt` 和 `dm-verity` 完整集成到星绽内核中，包括启动参数解析、BIO 路径适配、根文件系统切换。
- **内核启动自检机制**：实现了基于 ext2 超级块魔数（`0x53EF`）的启动时自检（`self_test_ext2_mag`）验证加解密路径正确性（补丁 0002）。
- **五阶段渐进式保护路径**：设计并实现了从 Plain 到 Combined（dm-crypt + dm-verity 叠加）的渐进式安全保护路径。

- **性能优化：**实现了认证后哈希块缓存（吞吐 +24.4%）、完成 BIO 数据复用、有界临时缓冲池（scratch pool）和 AES-NI 硬件加速。
- **Linux 兼容控制面：**实现了 `/dev/mapper/control` 的 Linux `dm_ioctl` 兼容接口，支持 `cryptsetup` 动态创建/删除映射器。

## 7.4 AI 工具使用声明

本项目在开发过程中，严格遵守比赛关于“允许参赛学生合理使用 AI 工具”的规定。

### 7.4.1 使用的 AI 工具及大模型

表 7.2: AI 工具使用详情

| 工具名称 | 相关大模型 | 具体使用场景与辅助内容 |
|---|---|---|
| OpenAI Codex | GPT-5 | 1. 阅读和梳理星绽框架、`dm-crypt`、`dm-verity` 相关底层代码；<br>2. 辅助分析 BIO 数据路径与编译错误；<br>3. 生成或修改部分 Rust、Shell 测试和构建脚本；<br>4. 协助整理参赛文档表述。 |
| 通义千问（Qwen） | Qwen-Max | 技术文档的深度润色，确保中英文专业术语的准确性与一致性。 |
| 豆包（Doubao） | 豆包大模型 | 答辩表述的逻辑优化，整理技术答辩常见问题及应对策略。 |
| WorkBuddy | 腾讯混元 Hy3、DeepSeek | 辅助执行测试、完整内核构建和 QEMU 验证并解读日志。 |

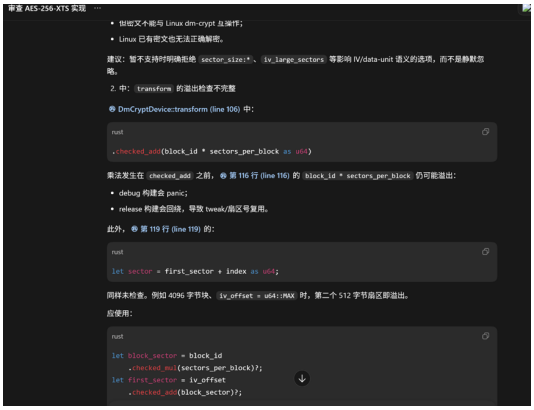
### 7.4.2 AI 交互记录

以下是开发过程中与 AI 工具的关键交互截图，证明 AI 仅作为辅助工具，开发逻辑由团队主导。

## OpenAI Codex 交互记录（GPT-5）

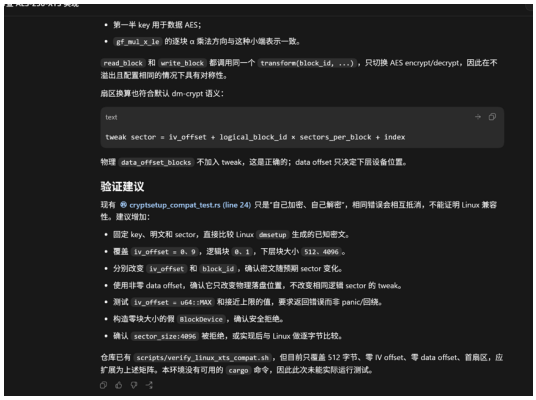



(a) 代码分析与框架理解




(b) BIO 路径辅助分析

图 7.1: OpenAI Codex 交互记录（1/4）




(a) Rust 测试脚本生成

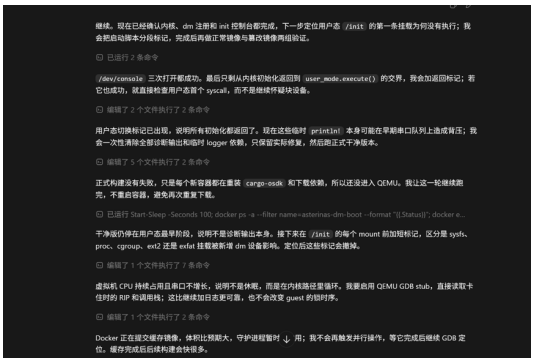



(b) Shell 构建脚本辅助

图 7.2: OpenAI Codex 交互记录（2/4）

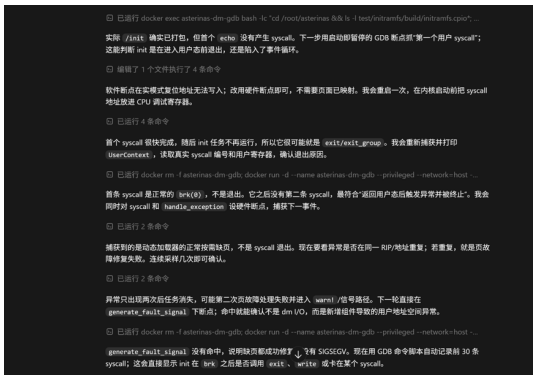



(a) QEMU 日志解读




(b) 编译错误调试

图 7.3: OpenAI Codex 交互记录（3/4）




(a) 内核 panic 追踪




(b) 文档表述整理

图 7.4: OpenAI Codex 交互记录（4/4）

## 通义千问（Qwen）交互记录




(a) 技术文档润色




(b) 专业术语校对

图 7.5: 通义千问（Qwen）交互记录

**豆包（Doubao）交互记录**




图 7.6: 豆包（Doubao）答辩表述优化与 FAQ 整理

### 7.4.3 验证机制与责任声明

为确保代码与文档的质量及原创性，项目组采取了以下严格措施：

1. **人工审查**：所有 AI 生成的代码片段和文档内容，均经过团队成员的人工 Peer Review。
2. **交叉验证**：代码正确性通过严格的单元测试、与 Linux dm-crypt 的逐字节对照、星绽内核构建及 QEMU 端到端运行进行了多重验证。
3. **责任承担**：最终的技术方案、代码取舍、架构决策和提交材料均由参赛团队全权负责。

## 7.5 Git 提交与开发迭代记录

### 7.5.1 提交概览

以下为项目主要 Git 提交记录（基于主仓库历史）：

表 7.3: Git 提交记录概览

| 序号 | 提交时间 | Commit Hash | 提交说明 | 变更统计 |
|---|---|---|---|---|
| 1 | 06-23 14:15 | 6bb9bc9 | 初始提交，仅源码 | 44 文件，+2500 行 |
| 2 | 06-23 22:36 | 40a5217 | Linux 兼容 Device Mapper 实现 | 18 文件，+1923/-61 行 |
| 3 | 06-23 22:58 | c72462b | 验证 Asterinas dm BIO 路径 | 6 文件，+280/-12 行 |
| 4 | 06-24 01:33 | 2f8a671 | 接入受保护根文件系统 | 12 文件，+186/-25 行 |
| 5 | 06-24 01:42 | a0dbb0f | 补充 AI 工具使用说明 | 1 文件，+4 行 |
| 6 | 06-24 01:48 | 84970aa | 文档中文化 | 6 文件，+118/-113 行 |
| 7 | 06-29 21:06 | f587893 | 添加 LaTeX 技术文档、PDF 与答辩 PPT | 多文件 |
| 8 | 06-30 17:08 | c22b8b4 | 添加演示视频入口 | 1 文件 |
| 9 | 07-14 18:44 | 5f345a1 | 完成安全存储性能对照与验收证据 | 多文件 |

此外，星绽内核集成补丁（asterinas_port/patches/）包含更细粒度的演进历史：

- 0001：组件集成与目标定义
- 0002：BIO 路径验证与自检（EXT2 魔数）
- 0003：根文件系统 pivot_root 切换
- 0004：Runtime 控制面与 cryptsetup 兼容
- 0005：性能优化（缓存、缓冲池、基准）
- 0006：性能阶段归因与正式合同

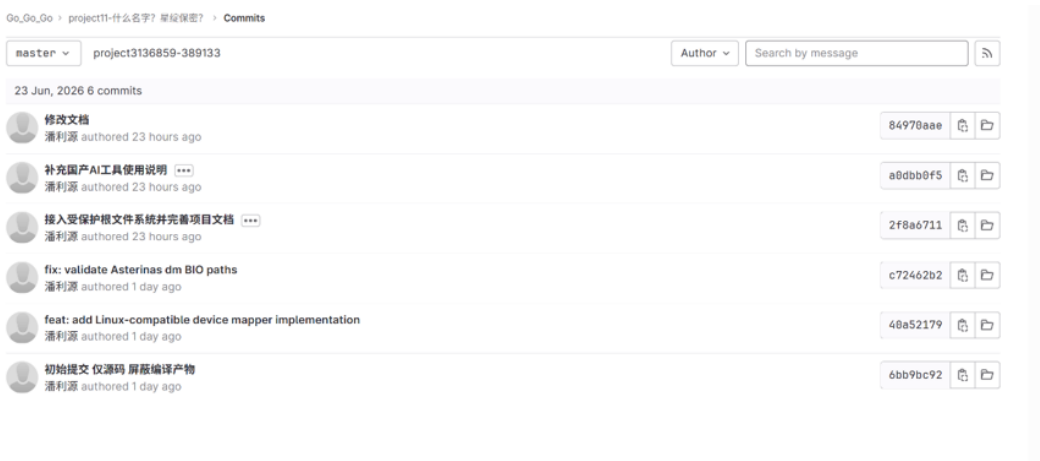



图 7.7: GitLab 仓库提交历史截图

### 7.5.2 关键迭代节点说明

1. **基础架构与核心逻辑**（Commit `40a5217`）：完成了 `BlockDevice` trait 的抽象，实现了纯 Rust 的 AES-256-XTS 加密与 SHA-256 默克尔树校验。
2. **内核集成与 BIO 路径打通**（Commit `c72462b`、`2f8a671`）：将用户态逻辑移植至星绽内核，成功实现了基于 `pivot_root` 的受保护根文件系统切换。
3. **性能优化与验收证据**（Commit `5f345a1`）：完成安全存储性能对照，包括正式 256 MiB 五轮基准、Linux guest 同条件对照、启动时间测量和哈希缓存 A/B 验证。
4. **文档与合规性完善**（Commit `84970aa`、`f587893`）：全面梳理技术文档，完成中文化翻译，补充 AI 工具使用声明，并添加 LaTeX 技术文档和答辩 PPT。

## 7.6 小结

本章严格按照全国操作系统设计赛的技术方案要求，全面披露了本项目的代码来源、开源协议合规性、借鉴基础与增量贡献、AI 工具使用声明以及 Git 提交与开发迭代记录。所有引用和借鉴的内容均已明确标注来源，所有 AI 辅助生成的内容均经过人工审查和验证。

# Chapter 8

# 用户手册与部署指南

## 8.1 概述

本章为开发者提供完整的部署指南，帮助用户从零开始搭建开发环境、编译星绽内核、配置启动参数并验证 dm-crypt 与 dm-verity 的功能。所有步骤均已在 Ubuntu 24.04 LTS 上验证通过。

## 8.2 运行环境准备

### 8.2.1 宿主机要求

推荐使用以下环境进行开发和测试：

- **操作系统**：Ubuntu 24.04 LTS（或 Windows WSL2 上的 Ubuntu 24.04）
- **架构**：x86_64
- **内存**：至少 8 GiB
- **磁盘空间**：至少 20 GiB
- **CPU**：支持硬件虚拟化（VT-x/AMD-V）和 AES-NI（推荐）

### 8.2.2 依赖工具安装

表 8.1: 开发环境依赖

| 依赖项 | 版本要求 | 用途 |
|---|---|---|
| Rust / Cargo | 1.75.0+ | 核心语言工具链 |
| QEMU | 8.2.2+ | 硬件虚拟化 |
| Docker | 29.4.3+ | 容器化构建环境（仅用于内核构建） |
| 星绽镜像 | asterinas/asterinas:0.17.0-20260618 | 官方工具链 |
| Clang / LLVM | 18.0.0+ | 内核编译 |

Ubuntu 24.04 上可通过以下命令安装基础依赖：

```
sudo apt update
sudo apt install -y build-essential clang lld qemu-system-x86 docker.io
```

Rust 工具链安装：

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
rustup default stable
```

## 8.3 快速开始：用户态演示

本节演示如何在宿主机上直接运行 dmctl 工具，无需启动虚拟机即可验证核心功能。

### 8.3.1 克隆仓库与运行测试套件

```
git clone <repository-url>
cd OS-
cargo test
```

预期输出：所有测试通过（当前 13+ 个测试，包括 crypt_test、verity_test、file_device_test、cryptsetup_compat_test 等）。

### 8.3.2 演示透明加密（dm-crypt）

```
cargo run -p dmctl -- demo-crypt
```

预期输出：

- plain: 显示可读明文
- raw-hex: 显示十六进制密文，与明文不同
- read: 显示恢复的明文，与原始明文一致
- ciphertext_differs: true

### 8.3.3 演示完整性校验（dm-verity）

```
cargo run -p dmctl -- demo-verity
```

预期输出：

- 输出 root-hash: <64-char-hex>
- 篡改后读取报告 tamper_detected: true block=1

### 8.3.4 演示组合栈（dm-crypt + dm-verity）

```
cargo run -p dmctl -- demo-stack
```

预期输出：显示两层映射器（crypt-root、verified-root）及其安全能力（confidentiality、integrity、read_only），并成功读取数据。

### 8.3.5 手动命令示例

#### 初始化与原始 I/O

创建测试镜像（8 块，每块 64 字节）：

```
cargo run -p dmctl -- init-image target/demo.img 8 64
```

直接读取底层（绕过映射器）：

```
cargo run -p dmctl -- raw-read-hex target/demo.img 64 1
```

篡改指定块内的字节（测试 verity）：

```
cargo run -p dmctl -- tamper target/demo.img 64 1 0 ff
```

#### 加密命令（dm-crypt）

加密写入：

```
cargo run -p dmctl -- crypt-write target/demo.img 64 my-secret-key 1 "hello dm-cryp
```

通过映射器解密读取：

```
cargo run -p dmctl -- crypt-read target/demo.img 64 my-secret-key 1
```

使用 Linux 兼容表格式填充密文（用于交叉验证）：

```
cargo run -p dmctl -- crypt-table-fill target/demo.img 512 \
  "aes-xts-plain64 <64-byte-hex-key> 0 ignored 0" 0 5a 1
```

#### 完整性命令（dm-verity）

生成根哈希：

```
root=$(cargo run -p dmctl -- verity-root target/demo.img 64)
```

验证完整性：

```
cargo run -p dmctl -- verity-verify target/demo.img 64 "$root"
```

为星绽内核格式化镜像（追加 on-disk 哈希树）：

```
cargo run -p dmctl -- verity-format-dm target/demo.img 1024
```

#### 微基准测试

```
cargo run -p dmctl -- benchmark 4096 5
```

输出 CSV 格式的吞吐和延迟数据，对比 plain / AES-XTS / verity 三种配置。

## 8.4 星绽内核集成

### 8.4.1 准备 Asterinas 源码

在宿主机上拉取星绽源码并切换到基线 commit：

```
git clone https://github.com/asterinas/asterinas.git
cd asterinas
git checkout c6284a9106f5a4c87deb7c8a990af6211dc5f540
```

### 8.4.2 应用 Device Mapper 补丁

将本项目中的补丁序列应用到 Asterinas 源码：

```
git am /path/to/OS-/asterinas_port/patches/0001-feat-dm-integrate-crypt-and-verity-
       /path/to/OS-/asterinas_port/patches/0002-fix-dm-validate-crypt-and-verity-BI
       /path/to/OS-/asterinas_port/patches/0003-rootfs-switch.patch \
       /path/to/OS-/asterinas_port/patches/0004-Rust-Device-Mapper.patch \
       /path/to/OS-/asterinas_port/patches/0005-dm.patch \
       /path/to/OS-/asterinas_port/patches/0006-dm-performance-attribution.patch
```

补丁说明：

- 0001：组件集成与目标定义（aster-dm 注册、启动参数）
- 0002：BIO 路径验证与自检（EXT2 魔数 0x53EF）
- 0003：根文件系统 pivot_root 切换
- 0004：Runtime 控制面与 cryptsetup 兼容
- 0005：性能优化（hash cache、scratch pool、基准）
- 0006：性能阶段归因与正式合同

### 8.4.3 构建内核

使用官方 Docker 容器构建（推荐）：

```
docker run --rm --privileged --network=host \
  -v asterinas-src:/root/asterinas \
  -w /root/asterinas \
  asterinas/asterinas:0.18.0-20260618 make kernel
```

启用 AES-NI 硬件加速构建：

```
docker run --rm --privileged --network=host \
  -v asterinas-src:/root/asterinas \
  -w /root/asterinas \
  -e OSDK_ENABLE_AES_NI=1 \
  asterinas/asterinas:0.18.0-20260618 make kernel
```

构建产物位于 /root/asterinas/target/osdk/aster-kernel-osdk-bin.iso。

### 8.4.4 构建受保护的根文件系统镜像

在项目根目录运行：

**dm-crypt 加密镜像：**

```
sudo bash scripts/build_crypt_rootfs.sh
```

镜像生成于 target/crypt-rootfs.img，并附带环境变量文件 target/crypt-rootfs.img.env
包含密钥和配置参数。

**dm-verity 校验镜像：**

```
bash scripts/build_verity_rootfs.sh
```

镜像生成于 target/verity-rootfs.img，并附带环境变量文件 target/verity-rootfs.img.e
包含根哈希、数据块数、哈希起始块号等。

## 8.5 系统运行与参数配置

### 8.5.1 QEMU 启动命令

基本启动命令（使用官方内核 ISO）：

```
qemu-system-x86_64 \
  -accel kvm -cpu host -smp 1 -m 8G \
  -machine q35,kernel-irqchip=split \
  -cdrom target/osdk/aster-kernel-osdk-bin.iso \
  -drive file=target/verity-rootfs.img,format=raw,if=virtio \
  -nographic -display none
```

对于已应用补丁的内核，使用 `asterinas/` 目录下的启动脚本更方便：

```
cd asterinas
DM_IMAGE=../target/verity-rootfs.img ENABLE_KVM=1 \
  bash tools/run_dm_verity.sh
```

### 8.5.2 内核启动参数配置

**启用 dm-verity**（保护根文件系统完整性）：

```
rootfs_verity.scheme=dm-verity
rootfs_verity.device=vdc
rootfs_verity.name=dm-verity0
rootfs_verity.hash=6db653b69a094065756dc2770d3af8880ec9beed29df41b5bb2fb2e11b2d49f3
rootfs_verity.data_blocks=65536
rootfs_verity.hash_start=65536
rootfs_verity.cache_blocks=256
```

**启用 dm-crypt**（加密根文件系统）：

```
dm_crypt.device=/dev/vda
dm_crypt.name=dm-crypt0
dm_crypt.key=000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f202122
dm_crypt.scratch_pool=32
```

**启用性能阶段统计**（仅基准测试）：

```
dm.perf_stats=1
```

启动流程的关键阶段（如图 8.1 所示）：

1. QEMU 加载星绽内核和 initramfs。
2. 内核解析命令行参数，构建配置。
3. `aster-dm` 组件初始化映射器链（`VirtioBlkAdapter` → `DmCryptDevice` / `DmVerityDevice`）。

4. 执行 `self_test_ext2_magic` 自检（检查 EXT2 魔数 `0x53EF`），失败则 panic。
5. initramfs 使用 `dm-root` 模式执行 `pivot_root` 切换到受保护根文件系统。
6. 系统进入用户态，串口输出 `PASS system root switched to protected mapper`。

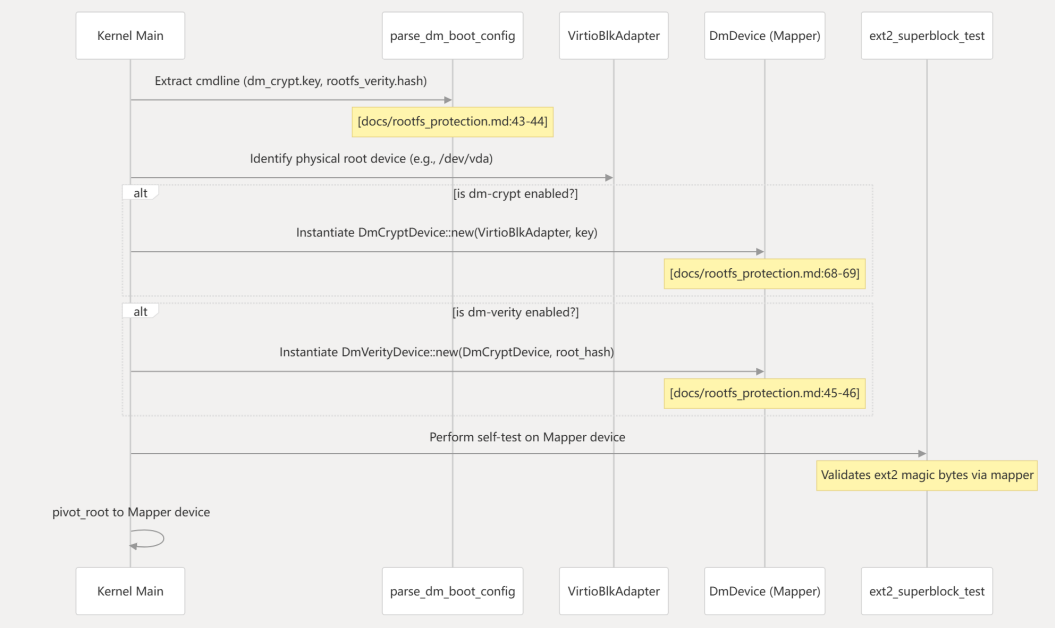



图 8.1: 根保护初始化序列图

## 8.6 基准测试与性能验证

### 8.6.1 正式性能测试（256 MiB 合同）

一键执行完整基准（包含镜像构建和五轮 KVM 测试）：

```
sudo bash scripts/run_formal_dm_benchmark.sh --build-images
```

该命令会：

1. 构建 crypt 和 verity 镜像（256 MiB 数据区，264 MiB 容器）。
2. 在 KVM 中交替运行 raw/dm-crypt/dm-verity 各五轮（2s ramp + 10s runtime）。
3. 生成统一三行结果表（`raw-crypt-verity.csv`）。
4. 生成阶段归因 CSV（`dm-crypt-stages.csv`、`dm-verity-stages.csv`）。

输出位于 `target/benchmark/formal-256m/`。

### 8.6.2 启动时间测量

执行五轮交替冷启动测试：

```
python3 scripts/benchmark_boot_time.py \
  --plain-command "bash scripts/run_asterinas_boot_once.sh plain" \
  --protected-command "bash scripts/run_asterinas_boot_once.sh dm-verity" \
  --plain-marker "PASS system root switched to protected mapper" \
  --protected-marker "PASS system root switched to protected mapper" \
  --runs 5 --output target/benchmark/boot-time.csv
```

### 8.6.3 用户态微基准测试

```
bash scripts/benchmark.sh
```

输出 CSV 文件包含 plain / AES-XTS / verity 三种配置的吞吐量对比。

## 8.7 高级配置与调优

### 8.7.1 dm-crypt 临时缓冲池调优

启动参数 `dm_crypt.scratch_pool` 控制临时缓冲区池大小（默认 32）。设为 0 可关闭池，便于 A/B 性能对照：

```
dm_crypt.scratch_pool=0
```

### 8.7.2 dm-verity 哈希缓存调优

启动参数 `rootfs_verity.cache_blocks` 控制已认证哈希块缓存大小（默认 256）。增大容量可提高缓存命中率，但增加内存占用：

```
rootfs_verity.cache_blocks=512
```

### 8.7.3 内核阶段统计

开启阶段统计（用于性能归因）：

```
dm.perf_stats=1
```

统计信息在每轮 fio 前后通过 `/proc/dm-performance-stats` 输出，包含以下阶段：

- `validation_ns`、`scratch_ns`、`copy_ns`、`crypto_ns`、`lower_wait_ns`（dm-crypt）
- `data_sha256_ns`、`cache_lookup_ns`、`hash_wait_ns`、`hash_sha256_ns`、`digest_compare_ns`、`cache_insert_ns`（dm-verity）

## 8.8 故障排查

### 8.8.1 常见问题

- **内核构建失败**：确保 Docker 容器使用正确的 Asterinas commit（`c6284a9`）并已应用全部 6 个补丁。检查 `asterinas_port/README.md` 中的补丁顺序。
- **QEMU 无法启动**：检查是否安装了 KVM 支持（`/dev/kvm` 是否存在），并确保镜像路径正确。对于 WSL2 环境，需要确认 `/dev/kvm` 可用。
- **dm-verity 启动失败**：确认根哈希与镜像匹配。使用 `dmctl verity-root` 重新生成根哈希，并与启动参数中的 `rootfs_verity.hash` 对比。检查 `data_blocks` 和 `hash_start` 参数是否与镜像一致。
- **dm-crypt 解密失败**：检查密钥是否正确（64 字节十六进制），且 `iv_offset` 和 `offset` 参数与加密时一致。查看串口日志确认 `backend` 输出（AES-NI 或 portable-software）。
- **性能数据不可复现**：确保使用 `release-lto` 构建，KVM 加速和 `CPU=host`，且 fio 合同完全一致（2s ramp + 10s runtime，交替五轮）。检查 `dm.perf_stats=1` 是否开启。
- **自检失败（EXT2 魔数不匹配）**：确认底层镜像确实是 ext2 格式，且文件系统位于镜像开头（无额外偏移）。对于 dm-crypt，确认 `offset` 参数为 0 或与镜像布局匹配。
- `cryptsetup` 无法控制：检查 `/dev/mapper/control` 是否存在，以及是否有权限访问。在 Asterinas 中，该设备由 `aster-dm` 组件在初始化时注册。

### 8.8.2 调试技巧

- 查看串口日志：在 QEMU 启动时使用 `-serial stdio` 或 `-serial file:serial.log`。
- 启用内核调试输出：启动参数中添加 `console=ttyS0` 并确保 initramfs 使用 `/dev/console`。
- 阶段统计验证：使用 `dm.perf_stats=1` 并检查 `/proc/dm-performance-stats`。
- 验证镜像完整性：使用 `dmctl verity-verify` 在用户态测试镜像。
- 检查补丁状态：在 Asterinas 源码中运行 `git log --oneline` 确认补丁已应用。

## 8.9 小结

本章提供了完整的用户手册与部署指南，涵盖了从环境准备、代码编译、镜像构建到系统运行的全流程，并给出了基准测试、调优建议和故障排查方法。通过遵循本章的步骤，开发者可以快速验证核心功能并部署星绽内核到 QEMU/KVM 环境

中。

# Chapter 9

# 研发过程与问题解决记录

## 9.1 概述

本章记录项目开发过程中遇到的核心技术难题及解决方案，并详细说明将 dm-crypt 和 dm-verity 集成到星绽内核的基础补丁序列。这些补丁构成了 `aster-dm` 组件的完整演进历史，可供评审逐行审计。

## 9.2 核心技术难题及解决方法

### 9.2.1 Rust 所有权与 I/O 队列管理

**问题描述：**在实现 `DmCryptDevice` 的 I/O 路径时，需要同时处理多个扇区的加解密操作。Rust 的所有权模型要求每个扇区的数据缓冲区有明确的所有者，导致借用检查器报错。尤其是在异步 BIO 完成回调场景中，多个请求共享同一状态槽时，生命周期管理变得复杂。

在早期的内核实现中，BIO 完成回调直接执行 AES-XTS 解密和上层完成，但回调上下文可能持有锁，导致潜在的锁竞争或死锁。

**解决方案：**

- 采用 `Arc<Mutex<Vec<u8>>` 在多个扇区操作之间共享缓冲区。
- 利用 `try_borrow_mut` 避免死锁，在锁获取失败时返回错误而非等待。
- 对于热路径，使用有界临时缓冲池（`scratch_pool`）预分配 `Vec`，通过池的 `take` 和 `recycle` 方法复用，避免在 I/O 路径中频繁分配和释放。
- 在实验性异步路径中，使用 `Arc<Mutex<Option<SubmittedBio>>>` 确保每个下层 BIO 的完成回调只触发一次上层完成，通过 `take()` 防止重复完成。
- 最终将解密操作从 VirtIO 硬中断回调迁移到专用内核线程（实验分支），减少锁持有时间。

### 9.2.2 星绽 OSTD 框架 API 适配

**问题描述：**星绽 OSTD 框架的块设备接口与预期不符，特别是 DMA 切片的访问方式。早期实现错误地使用虚拟地址操作 BIO 数据，导致加密和解密无法正确读写 DMA 物理缓冲区。

具体表现为：在对 `virtio-blk` 设备进行加密写入后，直接读取底层镜像发现数据未被正确加密，或者解密读回时数据损坏。

**解决方案：**

- 重构 `VirtioBlkAdapter`，直接调用 DMA 切片接口（`segment.inner_dma_slice()`）进行读写，而非通过普通内存访问。
- 使用 `BioSegment::alloc_bytes` 分配精确长度的下层 I/O 段，确保与加密扇区大小匹配，避免段对齐错误。
- 引入 `self_test_ext2_magic` 自检机制，在启动时验证解密路径是否正确（检查 EXT2 魔数 `0x53EF`），确保 DMA 数据路径无误。
- 严格遵循 Rust 安全准则使用 `unsafe` 代码块，仅局限于调用底层驱动接口，并在安全抽象层之上构建加密逻辑。

### 9.2.3 DMA 切片与 BIO 路径验证

**问题描述：**加密逻辑操作的是 BIO 段的虚拟地址，而非实际的 DMA 物理缓冲区。在星绽内核中，BIO 段的数据可能驻留在 DMA 缓冲区中，不能通过普通内存访问直接操作。

此外，BIO 的读写方向（`BioDirection::FromDevice` 和 `ToDevice`）需要正确设置，否则 DMA 切片的数据方向错误会导致数据损坏。

**解决方案：**

1. 使用 DMA 切片接口访问 BIO 段（`segment.inner_dma_slice()`），而非 `segment.read_bytes/w`
2. 确保加密操作直接读写 DMA 缓冲区，而非通过中间 `Vec` 拷贝（除必要的加解密转换外）。
3. 写路径：从调用方 DMA 段复制明文到临时缓冲区，加密后写入下层 BIO 的 DMA 段。
4. 读路径：从下层 DMA 段复制密文到临时缓冲区，解密后写回调用方的 DMA 段。
5. 引入自检机制验证解密路径，补丁 0002 中实现 `self_test_ext2_magic` 检查 EXT2 魔数，确保读写方向和 DMA 访问正确。

### 9.2.4 根文件系统切换（pivot_root）与映射器生命周期

**问题描述：**Asterinas 的 `pivot_root` 要求当前根文件系统具有挂载父节点，而 initramfs 的特殊 rootfs 挂载不具备此条件，直接切换会失败。

此外，映射器设备在 pivot_root 后需要保持可用，且 initramfs 的工具链（如 poweroff）在切换后仍需可访问。

**解决方案：**

1. 采用两阶段 dm-root 模式（补丁 0003）：
   - 第一阶段：initramfs 先将自身根目录通过 mount --bind 挂载到 /bootstrap-root，同时绑定挂载 /dev 到 /bootstrap-root/dev。
   - 使用 chroot /bootstrap-root 进入该挂载点，然后执行第二阶段 init（传递 dm-root-stage2 参数）。
   - 第二阶段：将映射器设备挂载到 /newroot，执行 pivot_root /newroot /newroot/.initramf 将受保护根文件系统切换为系统根。
2. 切换后，initramfs 工具链保留在 /.initramfs 目录下，确保关机等操作仍然可用。
3. 映射器设备在 pivot_root 前已注册到块设备子系统，切换后继续存在，不受根文件系统切换影响。
4. 在切换前验证 /etc/dm-verity-proof 文件是否存在，确保受保护根文件系统正确挂载。

## 9.3 内核集成补丁说明

以下补丁序列位于 asterinas_port/patches/，基于 Asterinas commit c6284a9106f5a4c87deb

### 9.3.1 补丁 0001：组件集成与目标定义

**目的：**将 aster-dm 组件引入星绽构建系统，实现基础 mapper 框架。

**实现细节：**

- **组件注册：**将 aster-dm 添加到 Cargo.toml（workspace members）和 Components.toml，使内核构建系统包含该组件。
- **分区扫描抑制：**在 aster-block 的 BlockDevice trait 中增加 supports_partition_scan 方法，允许 dm-verity 设备跳过分区扫描，避免误解析哈希区为分区表。
- **目标抽象：**定义 DmTarget 枚举（位于 device.rs），封装 Passthrough、Crypt（含 AesXtsCipher）和 Verity（含 VerityTarget）三种目标。
- **设备结构：**实现 DmDevice，持有下层设备引用和目标，通过 enqueue 方法分发 BIO 到对应的处理函数（process_passthrough、process_crypt、process_verity）。

- **核心模块：**引入 crypto.rs（AES-256-XTS 实现）、sha256.rs（纯 Rust SHA-256）、verity.rs（on-disk hash tree 校验）。

- **启动参数**：定义 dm_crypt.*（device、name、key）和 rootfs_verity.*（scheme、device、name、hash、data_blocks、hash_start）命令行参数解析。
- **initramfs 增强**：修改 initramfs 脚本以检测并挂载 /dev/vdc 上的 dm-verity mapper，验证 /etc/dm-verity-proof 文件。

### 9.3.2 补丁 0002：BIO 路径验证与自检

**目的**：修复 DMA 切片访问方向错误，并增加启动时自检机制。

**实现细节**：

- **DMA 切片访问**：使用 segment.inner_dma_slice() 替代原有 segment.read_bytes 和 segment.write_bytes，确保 DMA 缓冲区正确读写。
- **Ext2 超级块自检**：实现 self_test_ext2_magic 方法，检查偏移量 1080（1024+56）处的 EXT2 魔数 0x53EF，验证解密路径。
- **启动集成**：内核 init 函数在注册映射器后调用自检，失败则 panic 并输出 [dm-test] FAIL 日志。
- **错误处理**：增加 DmError::InvalidData 错误类型，用于自检失败。
- **测试脚本**：新增 tools/run_dm_crypt.sh 和 tools/run_dm_verity.sh，支持 KVM 测试，并在 initramfs 中输出 [dm-test] PASS/FAIL 日志。
- supports_partition_scan 默认返回 false，确保所有 dm 设备跳过分区扫描，避免误解析。

### 9.3.3 补丁 0003：根文件系统 Pivot_root 切换

**目的**：修改 initramfs 脚本，支持切换到受保护的根文件系统。

**实现细节**：

- **参数解析**：解析 dm-root 命令行参数，获取映射器设备名（如 /dev/dm-crypt0 或 /dev/dm-verity0）。
- **两阶段切换**：
  1. 第一阶段：initramfs 检测到 dm-root 参数后，将当前根目录绑定挂载到 /bootstrap-root，同时绑定挂载 /dev。
  2. 使用 chroot /bootstrap-root /init dm-root-stage2 <device> 进入第二阶段。
  3. 第二阶段：将映射器设备挂载到 /newroot，检查 /newroot/etc/dm-verity-proof 是否存在。
  4. 执行 pivot_root /newroot /newroot/.initramfs，将 initramfs 移动到 /.initramfs。

- **切换后验证**：检查 /etc/dm-verity-proof 和 /.initramfs 目录是否存在，确保受保护根文件系统正确激活。

- **initramfs 保留**：切换后 initramfs 工具链保留在 `/.initramfs` 目录，便于关机等操作（执行 `/.initramfs/bin/busybox poweroff -f`）。
- **启动脚本更新**：修改 `tools/run_dm_crypt.sh` 和 `tools/run_dm_verity.sh`，增加 `--init-args="dm-root /dev/<mapper> /.initramfs/bin/true"`。

### 9.3.4 补丁 0004：Runtime 控制面与 cryptsetup 兼容

**目的**：实现 Linux `dm_ioctl` 控制接口，支持动态创建/删除 mapper，为 `cryptsetup` 提供基础。

**实现细节**：

- **控制设备**：在 `/dev/mapper/control` 实现 `DmControlDevice`（位于 `kernel/src/device/misc/dm`）支持以下 ioctl：
  - `DM_VERSION`、`DM_LIST_VERSIONS`
  - `DM_DEV_CREATE`：创建 runtime mapper
  - `DM_TABLE_LOAD`：加载 crypt table（支持 `aes-xts-plain64`）
  - `DM_DEV_SUSPEND`：激活 mapper（resume）
  - `DM_DEV_STATUS`：查询状态
  - `DM_DEV_REMOVE`：删除 mapper
- `DmCryptTable` 解析增强：支持选项计数（`allow_discards`、`no_read_workqueue` 等）。
- **Runtime 映射器管理**：在 `runtime.rs` 中实现 `create_runtime_mapper`、`load_runtime_crypt_ta` `activate_runtime_mapper`、`remove_runtime_mapper` 函数，管理动态 mapper 生命周期（状态：Empty、Loaded、Active）。
- **设备节点创建**：成功激活后自动在 `/dev/mapper/` 下创建设备节点（通过 `add_node`）。

- **性能 procfs**：添加 `/proc/dm-crypto-backend`（显示 AES 后端）、`/proc/dm-verity-stats`（显示 verity 计数器）。
- **AES-NI 支持**：通过 `ostd::arch::x86::aesni` 模块，在保存/恢复 FPU 状态的前提下使用 AES-NI 指令；通过 `cpufeatures` 检测 CPU 支持。
- **密钥清除**：在 `remove_runtime_mapper` 和 `activate_runtime_mapper` 中清除密钥（`table.key.fill(0)`），避免密钥残留在内存中。

### 9.3.5 补丁 0005：性能优化与可配置基准

**目的**：加入认证后 hash cache、有界临时缓冲池、已认证路径捷径等优化，并提供可配置 fio 基准。

**实现细节**：

- **有界临时缓冲池**：在 CryptTarget 中实现 CryptScratchPool，支持 take 和 recycle，大小由启动参数 dm_crypt.scratch_pool 控制（默认 32）。
- **认证后 hash cache**：在 VerityTarget 中实现有界 FIFO 缓存（VerifiedHashBlockCache），只插入已完整认证的 hash block。
- **已认证路径捷径**：当缓存命中时，直接比对当前数据摘要与缓存块，跳过上层哈希重算，增加 auth_shortcuts 计数器。
- **BIO 数据复用**：对 4 KiB 对齐的读请求，直接复用已完成 BIO 的数据块，避免再次读取（verify_segments 方法）。
- **可配置基准**：tools/run_dm_crypt_bench.sh 和 tools/run_dm_verity_bench.sh 支持 DM_BENCH_* 环境变量，包括 rw、bs、qd、size、runtime、runs、ramp 等。
- test/initramfs/src/init 增加 dm-bench 模式，支持在 KVM 客体中自动运行 fio 矩阵并输出串口日志。

### 9.3.6 补丁 0006：性能阶段归因与正式基准合同

**目的**：添加细粒度阶段计时，固化正式 256 MiB 基准合同。

**实现细节**：

- **阶段统计**：在 device.rs 和 verity.rs 中埋点，使用 DmAtomicPerformanceStats 原子计数器记录以下阶段：
  - dm-crypt：validation、scratch、copy、crypto、lower_wait
  - dm-verity：validation、lower_wait、data_sha256、cache_lookup、hash_wait、hash_sha256、digest_compare、cache_insert
- /proc/dm-performance-stats：输出累计计数器，每轮 fio 前后读取，由汇总器（summarize_dm_performance_stages.py）计算差值。
- **正式合同**：run_formal_dm_benchmark.sh 强制：
  - 内核构建：release-lto
  - 虚拟化：KVM、CPU=host、1 vCPU、8 GiB
  - 块设备：virtio-blk，264 MiB 容器，前 256 MiB 数据区
  - fio：read、4 KiB、sync、direct=1、QD1
  - 时长：2s ramp + 10s runtime
  - 重复：raw/target 交替顺序，各五轮
  - 阶段统计：dm.perf_stats=1
- **metadata 校验**：汇总器严格核对 git commit、dirty 状态、镜像 SHA-256、fio 参数，不一致则拒绝生成结果表。
- **观察者开销校准**：run_dm_stats_calibration.sh 对比 stats=0/1 的吞吐差异，确保计时开销 ¡ 3%。

## 9.4 五阶段渐进式保护路径

为了确保系统稳定性和验证各层正确性，我们设计了五阶段渐进式测试路径：

表 9.1: 五阶段渐进式保护路径

| 阶段 | 名称 | 数据流 | 用途 |
|---|---|---|---|
| 1 | Plain | virtio-blk → ext2 | 基线性能和功能检查 |
| 2 | Passthrough | virtio-blk → PassthroughDevice → ext2 | 验证抽象层不引入数据损坏 |
| 3 | dm-verity | virtio-blk → DmVerityDevice → ext2 | 只读完整性校验 |
| 4 | dm-crypt | virtio-blk → DmCryptDevice → ext2 | 透明解密 |
| 5 | Combined | virtio-blk → DmCryptDevice → DmVerityDevice → ext2 | 全面保护（机密性 + 完整性） |

每阶段均通过 QEMU/KVM 端到端测试，并在启动日志中验证。组合路径在补丁 0004 中通过 `demo-stack` 命令展示，用户态可演示完整栈。

## 9.5 实验性优化探索

在正式基线之外，我们还进行了以下实验性探索：

### 9.5.1 异步 BIO 完成路径

实验分支 `codex/dm-async-bio` 将 dm-crypt 的下层 BIO 从同步 `submit_and_wait` 改为异步完成回调。实现要点：

- 使用 `Bio::submit_without_wait` 提交下层 BIO。
- 完成回调中执行 AES-XTS 解密、scratch 归还和上层 BIO 完成。
- 使用 `Arc<Mutex<Option<SubmittedBio>>>` 保证上层 BIO 只完成一次。

五轮 KVM 测试显示 raw 和 dm-crypt 吞吐同时提升约 49%，但 raw 归一化后的效率变化为 -0.62%，因此未合入基线。该实验证明了 I/O 路径仍有优化空间，但收益主要来自宿主机环境波动，非代码改动。

### 9.5.2 完成队列与专用 worker

后续分支 `codex/dm-completion-worker` 将解密从 VirtIO 硬中断回调迁到专用内核线程：

- 中断回调只记录完成状态并入队（无分配）。
- 专用 worker 在普通线程上下文执行解密和上层完成。

该改动使 32-job 并发从停滞变为五轮稳定完成。进一步批量出队（commit `6b3ca2407`）和每 CPU worker（commit `9a3483571`）优化分别取得：

- 批量出队（1 vCPU）：QD1 和 32-job 的配对 raw 保持率分别 +7.66% 和 +7.07%。
- 每 CPU worker（4 vCPU）：32-job 完成队列占比从 29.15% 降至 6.55%，p95 降低 43.62%；但 QD1 保持率下降 4.10%。

这些实验证据证明并发路径仍有优化空间，但因 QD1 有退化，暂不合入正式基线。

## 9.6 小结

本章详细记录了项目研发过程中遇到的四大核心技术难题及解决方案，深入阐述了 6 个基础内核集成补丁的设计思路和实现细节，并展示了五阶段渐进式保护路径和实验性优化探索。通过补丁 0002 的 Ext2 超级块自检机制和补丁 0003 的 `pivot_root` 切换，系统能够在启动时验证加解密路径的正确性，并安全地切换到受保护的根文件系统；后续功能与性能收尾以补丁 0004–0006 固化，最终形成可复核、可量化的完整技术交付。实验性优化成果为未来改进指明了方向。

# 附录 A

# 附录A：术语表

| 术语 | 定义 | 代码位置 |
| --- | --- | --- |
| BlockDevice | 表示任何可以读写固定大小块的存储介质的 trait 抽象 | crates/dm/src/block_device.rs |
| dm-crypt | 提供透明磁盘加密的设备映射器目标 | crates/dm/src/dm_crypt.rs |
| dm-verity | 提供透明完整性校验的设备映射器目标 | crates/dm/src/dm_verity.rs |
| HashTree | dm-verity 使用的默克尔树结构 | crates/dm/src/hash_tree.rs |
| AES-256-XTS | 磁盘加密的标准密码模式 | crates/dm/src/crypto.rs |
| plain64 IV | 使用 64 位扇区号作为 IV 的策略 | crates/dm/src/crypto.rs |
| IntegrityViolation | 哈希不匹配时返回的错误 | crates/dm/src/error.rs |
| Root Hash | 默克尔树的顶层哈希，信任锚 | crates/dm/src/hash_tree.rs |
| Tweak Key | XTS 密钥的后半部分，用于加密 IV | crates/dm/src/crypto.rs |
| IV Offset | 用于计算初始化向量的偏移值 | crates/dm/src/dm_crypt.rs |
| Data Offset | 物理设备开头用于元数据的扇区数 | crates/dm/src/dm_crypt.rs |
| DM_SECTOR_SIZE | 设备映射器的原子 I/O 单位（512 字节） | crates/dm/src/dm_crypt.rs |
| DmDevice | 星绽内核中实现 BIO 映射的块设备 | asterinas/kernel/comps/dm/src/devi |
| DM 启动参数 | crypt/verity 参数解析与 mapper 注册 | asterinas/kernel/comps/dm/src/lib |
| PassthroughDevice | 直接转发调用的包装器 | crates/dm/src/device_mapper.rs |
| aster-dm | 星绽内核中的 Device Mapper 组件 | asterinas/kernel/comps/dm/ |
| DmTarget | 封装 Crypt 或 Verity 逻辑的枚举 | asterinas/kernel/comps/dm/src/devi |
| self_test_ext2_magic | 启动时自检函数（检查 EXT2 魔数） | 补丁 0002 |
| pivot_root | 切换根文件系统的机制 | 补丁 0003 |
| dmctl | 用户态命令行工具 | crates/dmctl/src/main.rs |
| MemoryBlockDevice | 基于内存的后端 | crates/dm/src/block_device.rs |
| FileBlockDevice | 基于文件的后端 | crates/dm/src/block_device.rs |
| VirtioBlkAdapter | 星绽内核中 VirtIO 块设备适配器 | asterinas/kernel/comps/dm/src/devi |

| 术语 | 定义 | 代码位置 |
| --- | --- | --- |
| Scratch Pool | dm-crypt 有界临时缓冲池，用于复用加解密缓冲区 | asterinas/kernel/comps/dm/src/dev |
| 认证后哈希缓存 | 只缓存经过完整 Merkle 路径认证的哈希块 | asterinas/kernel/comps/dm/src/ver |
| DmStackBuilder | 用户态下组合 crypt/verity 层的构建器 | crates/dm/src/device_mapper.rs |

# 附录 B

# 附录B：dmctl 命令参考

## 初始化与原始 I/O 命令

| 命令 | 签名与示例 | 实现位置 |
| --- | --- | --- |
| init-image | **签名**：init-image ¡path¿ ¡blocks¿ ¡block_size¿<br>**示例**：dmctl init-image disk.img 1024 4096<br>**说明**：创建零填充文件用作虚拟块设备 | `dmctl/src/main.rs` |
| raw-read-hex | **签名**：raw-read-hex ¡path¿ ¡block_size¿ ¡block_id¿<br>**示例**：dmctl raw-read-hex disk.img 4096 0<br>**说明**：读取块以十六进制打印（绕过映射器） | `dmctl/src/main.rs` |
| tamper | **签名**：tamper ¡path¿ ¡block_size¿ ¡block_id¿ ¡offset¿ ¡hex_byte¿<br>**示例**：dmctl tamper disk.img 4096 0 10 ff<br>**说明**：修改块内字节（测试 verity） | `dmctl/src/main.rs` |

## 加密命令（dm-crypt）

| 命令 | 签名与示例 | 实现位置 |
| --- | --- | --- |
| crypt-write | 签 名： crypt-write ¡path¿ ¡block_size¿ ¡key¿ ¡block_id¿ ¡data_string¿<br>示 例： dmctl crypt-write disk.img 4096 mysecretkey 0 ”Hello” | `dmctl/src/main.rs` |
| crypt-read | 签 名： crypt-read ¡path¿ ¡block_size¿ ¡key¿ ¡block_id¿<br>示例：dmctl crypt-read disk.img 4096 mysecretkey 0 | `dmctl/src/main.rs` |
| crypt-table-fill | 签 名： crypt-table-fill ¡path¿ ¡block_size¿ ¡table_str¿ ¡start¿ ¡byte¿ ¡count¿<br>示 例： dmctl crypt-table-fill disk.img 4096 ”aes-xts-plain64 ¡key¿ 0 /dev/loop0 0” 0 aa 10 | `dmctl/src/main.rs` |

## 完整性命令（dm-verity）

| 命令 | 签名与示例 | 实现位置 |
| --- | --- | --- |
| verity-root | 签 名： verity-root ¡path¿ ¡block_size¿<br>示 例： dmctl verity-root disk.img 4096 | `dmctl/src/main.rs` |
| verity-verify | 签 名： verity-verify ¡path¿ ¡block_size¿ ¡root_hash_hex¿<br>示 例： dmctl verity-verify disk.img 4096 a1b2... | `dmctl/src/main.rs` |
| verity-format-dm | 签名： verity-format-dm ¡path¿ ¡data_blocks¿<br>示例： dmctl verity-format-dm disk.img 1024<br>说明：为星绽内核格式化镜像 | `dmctl/src/main.rs` |

## 演示与基准测试命令

| 命令 | 签名与示例 | 实现位置 |
| --- | --- | --- |
| demo-crypt | 签名：demo-crypt<br>示例：dmctl demo-crypt | `dmctl/src/main.rs` |
| demo-verity | 签名：demo-verity<br>示例：dmctl demo-verity | `dmctl/src/main.rs` |
| demo-stack | 签名：demo-stack<br>示例：dmctl demo-stack | `dmctl/src/main.rs` |
| benchmark | 签名：benchmark [blocks] [iterations]<br>示例：dmctl benchmark 4096 5 | `dmctl/src/main.rs` |

# 参考文献


[1] Linux Kernel Community. *Linux dm-crypt Documentation*, 2024. Linux dm-crypt 内核文档.

[2] Linux Kernel Community. *Linux dm-verity Documentation*, 2024. Linux dm-verity 内核文档.

[3] Milan Broz et al. cryptsetup - linux unified key setup (luks). https://gitlab.com/cryptsetup/cryptsetup, 2024. LUKS 和 dm-crypt 用户态工具.

[4] National Institute of Standards and Technology (NIST). Nist special publication 800-38e: Recommendation for block cipher modes of operation: The xts-aes mode for confidentiality on storage devices. https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38e.pdf, 2010. XTS-AES 存储加密模式标准.

[5] National Institute of Standards and Technology (NIST). Fips pub 180-4: Secure hash standard (shs). https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf, 2015. SHA-256 标准规范.

[6] Asterinas Team. 星绽操作系统 (asterinas). https://github.com/asterinas/asterinas, 2024. 访问日期：2026年8月.